

A Systematic Survey on Large Language Models for Evolutionary Optimization: From Modeling to Solving

Yisong Zhang, Ran Cheng, Guoxing Yi, and Kay Chen Tan

Abstract—Large language models (LLMs) are increasingly integrated with evolutionary computation to support optimization tasks¹. However, existing surveys typically examine isolated roles of LLMs and do not provide a unified view that connects optimization modeling with optimization solving. To address this gap, we systematically review recent developments through a workflow-oriented framework. First, we organize the literature into two primary stages: LLMs for optimization modeling and LLMs for optimization solving². Second, we divide the solving stage into three paradigms according to the role of the LLM: stand-alone optimizers, low-level components embedded in optimization algorithms, and high-level managers for algorithm selection and generation. Third, we analyze representative methods, identify their technical limitations, and clarify their relationships with traditional optimization approaches. We further substantiate this taxonomy through benchmark systematization, baseline comparisons, and practitioner-oriented guidance, and we review interdisciplinary applications across the natural sciences, engineering, and machine learning. Based on the resulting analysis, we identify research directions toward dynamic, self-evolving, and agentic optimization ecosystems. An up-to-date collection of related literature is maintained at <https://github.com/ishmael233/LLM4OPT>.

Index Terms—Large language models, evolutionary algorithms, optimization modeling, optimization solving.

I. INTRODUCTION

Optimization underpins complex decision-making across engineering design [1], economic planning [2], scientific discovery [3], and many other domains [4], [5]. A practical optimization workflow first abstracts a real-world problem into a mathematical model and then applies an optimization algorithm to solve that model. These algorithms can be broadly divided into exact and approximate methods [6]. Exact methods can provide global-optimality guarantees, but their computational cost often grows prohibitively with problem scale. Approximate methods, including heuristics [6], can address larger and more complex problems efficiently, but they generally do not guarantee convergence to a global optimum. Moreover, the No Free Lunch theorem [7] implies that no single algorithm performs best across all problem classes. Consequently, effective deployment often requires substantial expertise in algorithm configuration [8], selection [9], and

customized design [10]. This dependence on expert knowledge remains a major barrier to transferring optimization methods from research to practice.

Machine learning (ML) has been introduced into evolutionary computation to reduce this dependence on expert knowledge [11], [12]. At the component level, ML supports population initialization [13], fitness evaluation [14], operator selection [15], parameter control [16], and solution manipulation [17]. At the strategy level, it enables automated algorithm selection [18] and algorithm generation [19]. Most existing approaches rely on reinforcement learning [20] or supervised learning [21], [22] and are trained on narrowly defined problem distributions. As a result, their performance often degrades on unseen problems, while adaptation requires costly and time-consuming retraining. Thus, conventional learning-based methods reduce some forms of manual design but do not fully resolve the generalization challenge.

Large language models (LLMs) offer a complementary route because they combine broad pretrained knowledge with semantic understanding, compositional abstraction, and in-context learning [23]. These capabilities support zero-shot or few-shot transfer across heterogeneous tasks [24]. They are particularly relevant when objectives, constraints, or design criteria are expressed in natural language or structured symbols, and when the workflow requires readable candidate generation, explanation, or cross-domain priors, as in drug discovery [25] and finance [26].

Within optimization, LLMs can contribute to both modeling and solving. For modeling, they translate informal problem descriptions into formal mathematical formulations [27]. For solving, they can operate as stand-alone iterative optimizers for language-structured tasks [28] or as components that augment algorithmic search [29]. However, this integration remains at an early stage and exhibits clear limitations. Stand-alone LLM optimizers often underperform classical algorithms [30], while domain-specific fine-tuning depends on datasets that are costly to construct and validate [27]. These limitations indicate the need for a unified analysis of where LLMs are effective, how they should interact with traditional methods, and which research gaps remain.

To provide this analysis, we integrate a systematic literature review with benchmark systematization, controlled baseline comparisons, and practitioner-oriented guidance. The main contributions are summarized as follows:

- We adopt a modeling-to-solving perspective that covers the complete optimization workflow. Based on this per-

¹This survey primarily focuses on *evolutionary optimization*, i.e., optimization based on evolutionary computation. For brevity, we use the term *optimization* throughout to denote this scope.

²In this survey, the terms *optimization modeling* and *optimization solving* are used as concise forms of *optimization problem modeling* and *optimization problem solving*, respectively.

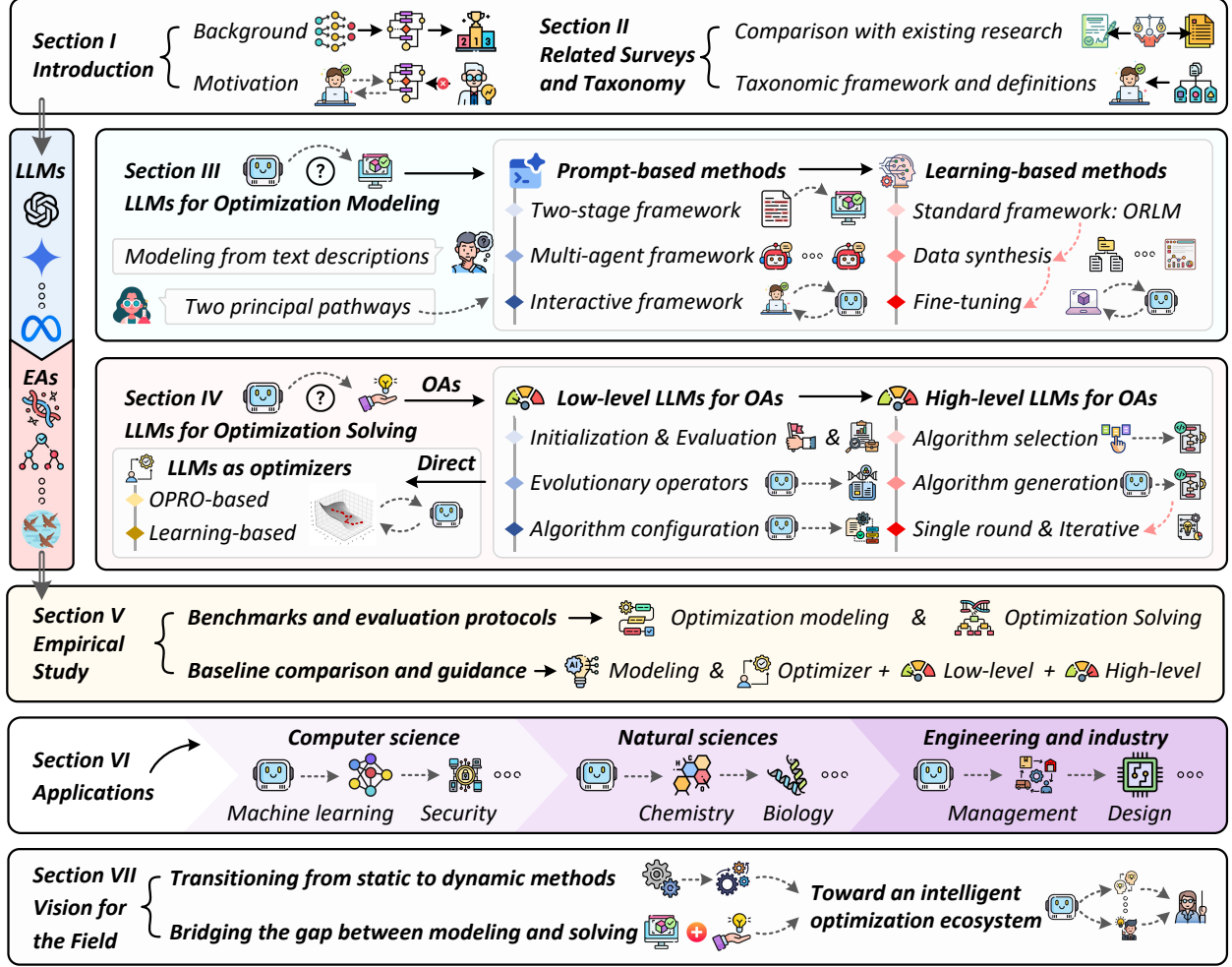


Fig. 1. Overall organization of this survey.

spective, we divide the field into optimization modeling and optimization solving, and further organize the solving stage into LLMs as optimizers, low-level LLM-assisted algorithms, and high-level LLM-assisted algorithm selection and generation. We ground the taxonomy empirically through data collection and controlled experiments across the modeling-to-solving spectrum. These analyses expose concrete failure modes, including limited numerical precision, weak dimensional scalability, and feasibility constraints. We translate the resulting evidence into practitioner-oriented method-selection guidance and identify research directions toward dynamic, self-evolving, and agentic optimization ecosystems.

Fig. 1 illustrates the organization of this survey. Section II reviews related surveys and defines the proposed taxonomy. Sections III and IV examine LLMs for optimization modeling and solving, respectively. Section V systematizes benchmarks, compares representative baselines, and derives practitioner-oriented guidance. Section VI reviews interdisciplinary applications. Finally, Section VII discusses future research directions, and Section VIII summarizes the main findings.

II. RELATED SURVEYS AND TAXONOMY

A. Survey Scope and Methodology

We conducted a structured literature search to obtain broad but technically focused coverage of the field. The search covered IEEE Xplore, the ACM Digital Library, Springer-Link, ScienceDirect, Google Scholar, and arXiv. We combined model-side keywords, including *large language model*, *LLM*, and *foundation model*, with task-side keywords, including *optimization*, *evolutionary algorithm/computation*, *metaheuristic*, *heuristic design*, *optimization modeling*, and *algorithm selection/generation*. The search period extended from January 2023 to September 2025, which covers the rapid expansion of the field following advances in prompting and in-context learning.

We applied two inclusion criteria. First, a study had to use one or more LLMs as an integral component of optimization modeling or solving. Second, it had to report a methodological contribution or an empirical evaluation. We excluded studies that mentioned LLMs without assigning them a concrete optimization role, as well as short abstracts and non-peer-reviewed notes that lacked sufficient methodological detail. After deduplication and manual screening of titles, abstracts,

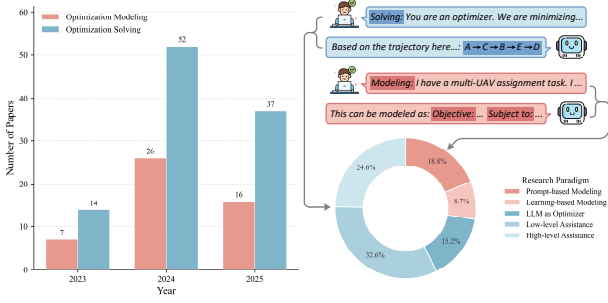


Fig. 2. Publication trends and paradigm distribution in the surveyed literature.

and full texts, we retained 152 representative papers as the corpus of this survey.

Because the literature is expanding rapidly, no survey can guarantee exhaustive coverage of every related study. We therefore aim to characterize the main technical landscape through a representative corpus of methodologically substantive work. Fig. 2 summarizes the publication timeline and the distribution of papers across the proposed taxonomy. The results show that most studies in the corpus were published during the final two years of the search period, which indicates a sharp recent increase in research on the integration of LLMs with optimization.

B. Related Surveys and Our Perspective

Several surveys have examined LLMs for optimization, as summarized in Table I. Wu *et al.* [31] and Huang *et al.* [32] adopt a bidirectional perspective but focus, on the optimization side, on LLMs as solvers or algorithm generators. Yu *et al.* [33] use a similar classification, whereas Chao *et al.* [34] focus on evolutionary algorithms (EAs) and emphasize LLMs as operators. Liu *et al.* [35] broadly review LLMs in algorithm design, and Ma *et al.* [12] survey MetaBBO with an emphasis on reinforcement learning.

Two structural gaps remain across these reviews. First, most studies focus on optimization solving, while optimization modeling receives limited attention or is treated as a peripheral application. Second, reviews of the solving stage commonly emphasize one role of the LLM without placing stand-alone optimization, component-level assistance, and high-level orchestration within a common framework. Consequently, the relationships among these complementary roles remain insufficiently characterized.

To address these gaps, this survey adopts a workflow-oriented perspective that follows the full optimization lifecycle from natural-language problem description to mathematical modeling and algorithmic solving. This perspective extends prior reviews in two respects. First, it treats optimization modeling as a primary stage rather than a peripheral application. Second, it organizes optimization solving as a spectrum of three roles: LLMs as optimizers, low-level LLM assistance, and high-level LLM assistance. This common frame makes the roles directly comparable and reveals an important missing link: current methods rarely form an end-to-end loop that connects automated modeling with LLM-assisted solving.

TABLE I
COMPARISON OF RELATED SURVEYS ON LLMs FOR OPTIMIZATION. THIS WORK SYSTEMATICALLY COVERS FOUR CATEGORIES: LLMs FOR MODELING (LM), LLMs AS OPTIMIZERS (LO), LOW-LEVEL LLMs FOR OPTIMIZATION ALGORITHMS (LL), AND HIGH-LEVEL LLMs FOR OPTIMIZATION ALGORITHMS (HL).

Ref.	Venue	LM	LO	LL	HL
Wu <i>et al.</i> [31]	TEVC, 2024		✓		✓
Huang <i>et al.</i> [32]	SWEVO, 2024		✓		✓
Chao <i>et al.</i> [34]	Research, 2024			✓	
Yu <i>et al.</i> [33]	arXiv, 2024		✓	✓	✓
Liu <i>et al.</i> [35]	arXiv, 2024		✓	✓	✓
Ma <i>et al.</i> [12]	TEVC, 2024		✓		✓
Our work	arXiv, 2025	✓	✓	✓	✓

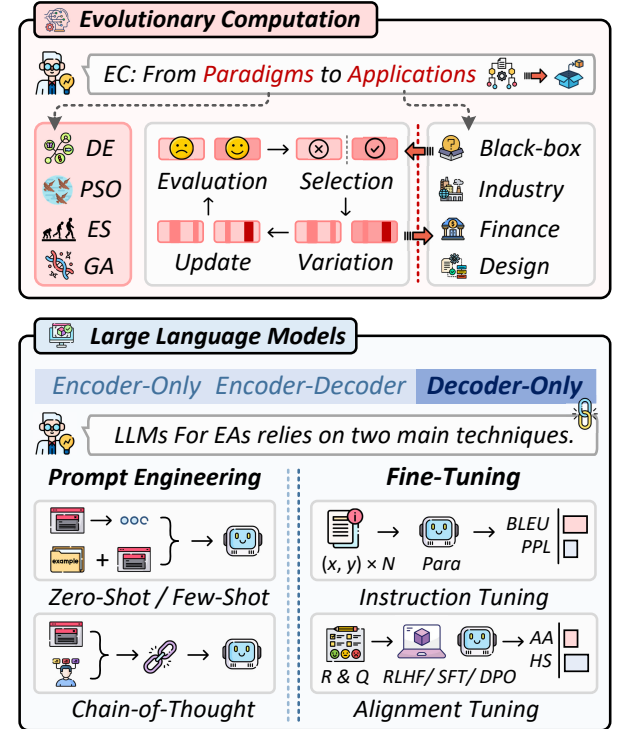


Fig. 3. Technological dependencies of LLMs for optimization, including EA paradigm and workflow, LLM architecture, and related enabling technologies.

Section VII revisits this gap. Fig. 3 summarizes the supporting technologies, including the EA workflow, LLM architectures, and related enabling methods, while Supplementary Document I provides further background on EAs and LLMs. Based on the same workflow-oriented perspective, Section V evaluates representative methods and derives practical method-selection guidance.

C. Taxonomy and Definitions

Based on this workflow-oriented perspective, we divide the field into two top-level categories: LLMs for optimization modeling and LLMs for optimization solving. We further divide the solving stage into three paradigms according to the role of the LLM. The four categories are defined as follows:

- **LLMs for Optimization Modeling** transform unstructured natural-language problem descriptions into mathematical optimization models that machines can interpret and solve. This category addresses the transition from ambiguous language to precise variables, objectives, and constraints. Existing methods are primarily prompt-based or learning-based. Prompt-based methods use designed instructions, structured workflows, or multiple agents and can be deployed without additional training. Learning-based methods use data synthesis and fine-tuning to improve reliability, but they introduce training and maintenance costs.
- **LLMs as Optimizers** use LLMs as stand-alone optimizers that iteratively generate candidate solutions through natural-language interaction, without embedding them in a traditional optimization framework. These methods exploit in-context learning and reasoning over previous solutions and feedback. Their direct formulation makes them simple to deploy, but their performance depends strongly on the underlying model and often scales poorly with numerical dimensionality.
- **Low-level LLM-assisted Optimization Algorithms** embed LLMs within established optimization algorithms to support specific components. Typical roles include population initialization, evolutionary variation, parameter control, algorithm configuration, and fitness evaluation. In this paradigm, the optimization algorithm retains control of the overall search, while the LLM contributes semantic priors, domain knowledge, or adaptive local decisions.
- **High-level LLM-assisted Optimization Algorithms** use LLMs for orchestration or design at the algorithm level. This category includes algorithm selection, in which an LLM chooses a suitable method from a portfolio, and algorithm generation, in which an LLM constructs or refines an optimization algorithm for a target task. The LLM therefore acts as a selector or designer rather than as an internal search component.

III. LLMs FOR OPTIMIZATION MODELING

Optimization modeling is the first computational stage of the optimization workflow, but it traditionally requires substantial domain and mathematical expertise. LLMs provide a mechanism for automating parts of this process by mapping natural-language specifications to formal variables, objectives, and constraints. This section reviews two main paradigms. Prompt-based methods, discussed in Section III-A, guide LLMs through designed instructions, two-stage workflows, multi-agent collaboration, or interactive refinement. Learning-based methods, discussed in Section III-B, fine-tune LLMs to generate mathematical formulations directly and commonly rely on synthetic training data. Section III-C then compares the limitations and deployment trade-offs of the two paradigms. Fig. 4 illustrates their main workflows, and Supplementary Document II summarizes representative studies.

A. Prompt-based Methods

Early work on automated optimization modeling attempted to reduce expert involvement by inferring problem structure

from feasible solutions or by using EAs. Pawlak *et al.* studied the inference of optimization models from solution samples [36] and the generation of constraints through grammatical evolution [37]. These studies established the feasibility of partial automation, but their performance depends strongly on sample quality and coverage [38]. Moreover, they primarily generate constraints rather than translate natural-language descriptions into complete mathematical models.

Subsequent research shifted toward the direct translation of unstructured language into mathematical optimization models. Ramamonjison *et al.* proposed OptGen [39] and initiated the NL4OPT competition [40], which decomposed modeling into named-entity recognition (NER) followed by model generation from annotated descriptions. Early systems used smaller language models, including BERT [41], [42] and BART [43], [44]. Almonacid *et al.* [45] later introduced a single-step LLM approach that prompts GPT-3.5 to generate a model and uses a solver to verify the result. Although this approach is effective on simple instances, a single generation step provides insufficient accuracy and robustness for more complex formulations.

To improve reliability, researchers developed two-stage frameworks that combine LLMs with specialized extraction modules. Holy Grail 2.0 [46] decomposes the workflow into entity-relation identification, problem formalization, and code generation. Li *et al.* [38] extended this structure to mixed-integer linear programming by fine-tuning a model to classify constraints, including logical constraints and binary variables that earlier methods often omitted. Similar two-stage workflows have been applied to energy management [47] and travel planning [48].

Two further extensions relax the fixed pipeline. Multi-agent frameworks assign complementary modeling roles to several LLM agents and use cross-checking to improve coverage and consistency. Interactive frameworks instead maintain a dialogue with the user so that the formulation can be revised as requirements and preferences change.

Multi-agent modeling commonly separates formulation, implementation, and verification across specialized agents [49]. The Chain-of-Experts (CoE) framework [50] coordinates 11 expert types through a conductor agent, constructs a forward chain of thought, and applies backward reflection to correct errors and inconsistencies. OptiMUS [51] and OptiMUS-0.3 [52] similarly use a conductor to coordinate modeling, programming, and evaluation. However, conductor-driven interaction can produce variable workflows and insufficient mathematical precision. To improve process control, ORMind [53] uses a structured, cognitively inspired workflow with counterfactual reasoning. Other studies focus specifically on verification: Mostajabdeh *et al.* [54] replace solver-based checking with peer verification among agents, while Talebi *et al.* [55] use independent reviewer agents to evaluate stochastic optimization models.

Interactive modeling addresses problems for which the initial specification is incomplete or changes during use. This requirement is common in conference scheduling, travel planning, and other preference-sensitive tasks. OptLLM [56] supports both single-shot and interactive inputs through a

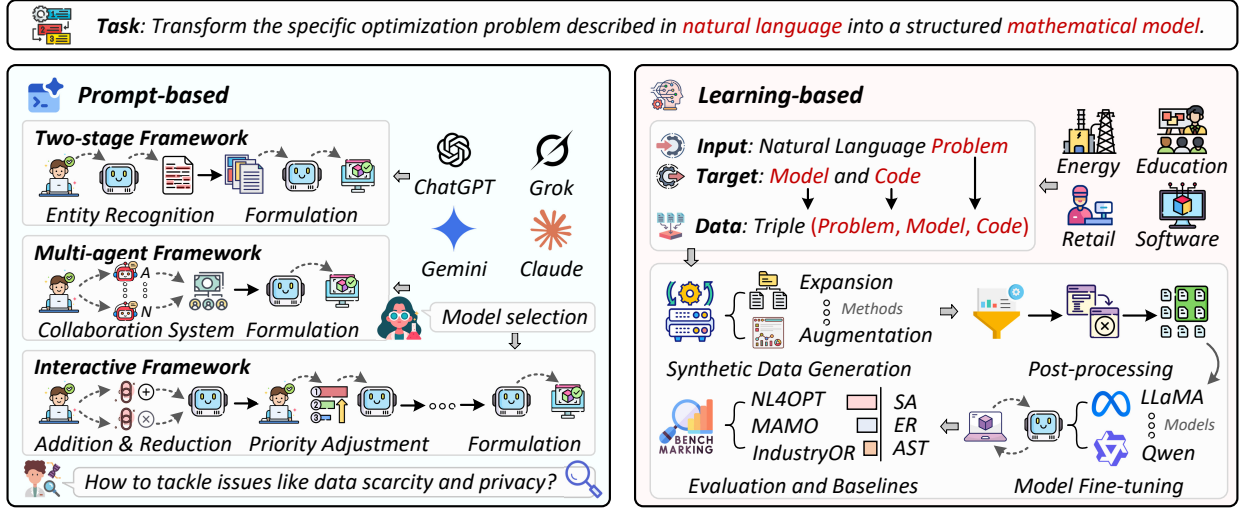


Fig. 4. Illustration of LLMs for optimization modeling. Approaches are divided into two categories: (i) prompt-based methods, which are typically implemented via two-stage prompting, multi-agent collaboration, or interactive frameworks; and (ii) learning-based methods, which generally follow a workflow involving data synthesis, model fine-tuning, and evaluation.

refinement module that progressively clarifies the problem description, followed by converter and response modules for modeling and verification. MeetMeta [57] requires the LLM to select among five actions after each user message, return a solution, explain unmet preferences, and accept subsequent modifications. Related work translates user priorities into constraints during dialogue, which allows users to examine trade-offs in applications such as ridesharing coordination [58].

Recent prompt-based research has also targeted generation and verification as separate technical bottlenecks [59]. For generation, Astorga *et al.* [60] combine LLMs with Monte Carlo Tree Search to explore a hierarchical hypothesis space through symbolic pruning and LLM-based evaluation. Li *et al.* [61] instead constrain generation with predefined structures and apply post-processing verification and correction. For verification, Huang *et al.* [62] use solver outputs to evaluate generated models and introduce MAMO, which extends NL4OPT with ordinary differential equations and more complex LP and MILP problems. Wang *et al.* [63] represent models as bipartite graphs and use a modified Weisfeiler-Lehman graph-isomorphism procedure to test equivalence in OptiBench. Zhai *et al.* [64] further introduce Quasi-Karp equivalence and the EquivaMap system, which combine LLM-generated variable mappings with lightweight verification for scalable equivalence detection.

Prompt-based methods remain bounded by the capabilities and deployment constraints of their underlying LLMs. Closed-source models such as the GPT series face two recurring limitations [27]. First, the scarcity of high-quality optimization-modeling data limits formal and domain-specific reasoning. Second, API-based deployment can expose sensitive problem descriptions in applications such as medical scheduling and defense planning. These limitations motivate learning-based methods that fine-tune open-source models to improve mathematical-formulation accuracy, preserve data control, and incorporate domain knowledge more directly.

B. Learning-based Methods

Learning-based methods fine-tune model parameters so that optimization-modeling knowledge is encoded in the LLM rather than supplied only through prompts. Early studies used fine-tuning for individual stages of the workflow. Ner4OPT [65] combines conventional NLP methods with LLMs for NER, while Li *et al.* [38] fine-tune a model to classify constraints and identify complex entity relations. These models remain auxiliary components within two-stage pipelines. LM4OPT [66] moves toward direct model generation through progressive fine-tuning of Llama-2-7B [67]. However, because it is trained only on NL4OPT, its performance remains below that of closed-source models such as GPT-4 [68].

ORLM [27] established a general workflow that combines data synthesis, instruction tuning, and evaluation. Its two-stage synthesis procedure produces aligned natural-language descriptions, mathematical models, and executable solver code. The expansion stage uses seed examples and GPT-4 to generate problems across scenarios and difficulty levels. The augmentation stage then modifies objectives and constraints and paraphrases problem descriptions, while matching correction and deduplication improve data consistency. The resulting data are used to fine-tune open-source models, including Mistral-7B [69] and DeepSeek-Math-7B-Base [70]. These models outperform single-step GPT-4 generation and prompt-based methods such as CoE [50] and OptiMUS [51] in the reported evaluations. Subsequent studies have largely followed this workflow while improving either data synthesis or fine-tuning.

One line of learning-based research focuses on synthesizing reliable and diverse training data [71]. ReSocratic [72] uses inverse generation: it first constructs mathematical optimization examples and then translates them into natural-language problem statements. OptMATH [73] uses bidirectional synthesis to control problem complexity and verifies generated pairs through forward modeling and rejection sampling. DualReflect [74] combines the diversity of forward generation with the re-

liability of inverse generation for dynamic programming problems. Other work improves annotation quality and curriculum structure. StructuredOR [75] adds detailed modeling-process annotations, including variable definitions, while Step-Opt [76] increases problem complexity iteratively and validates each generation step.

A parallel line of research improves the fine-tuning objective and training procedure [77]–[80]. LLMOPT [77] uses multi-instruction fine-tuning for problem formalization and solver-code generation, together with alignment and self-correction mechanisms that reduce hallucinations. SIRL [78] incorporates reinforcement learning and uses an external solver to provide verifiable rewards, which improves the factual and mathematical correctness of generated formulations. Amarasinghe *et al.* [79] adapt fine-tuning to business applications such as production scheduling and show that smaller, task-specific models can provide a cost-effective deployment option.

C. Challenges

Current methods demonstrate that LLMs can map natural-language descriptions to formal optimization models, but they do not eliminate the central trade-offs of automated modeling. CoE [50] and ORLM [27], for example, can produce complete formulations and identify constraints that may be implicit in the original description. However, prompt-based and learning-based methods differ substantially in deployment speed, data requirements, reliability, privacy, and maintenance cost. Their principal limitations are summarized as follows:

- **Prompt-based methods** can be deployed rapidly without constructing a large annotated dataset. However, their formulation accuracy is bounded by the formal reasoning and numerical capabilities of the pretrained model. Errors commonly arise when the problem requires precise symbolic structure, implicit constraint recovery, or numerical consistency. Reliance on external APIs also creates data-governance concerns in sensitive applications. Private deployment can reduce exposure risk, but it does not resolve the underlying reasoning limitations. Progress therefore requires stronger mathematical pretraining and hybrid architectures that connect LLM generation with symbolic parsers, solvers, or other external validators.
- **Learning-based methods** improve reliability and data control by internalizing domain-specific modeling knowledge through synthetic data and fine-tuning. However, data generation, verification, and retraining are expensive, particularly when they must be repeated across problem domains or model scales. The field also lacks established procedures for updating fine-tuned models without performance regression. Moreover, rapid improvements in general-purpose reasoning models can reduce the value of carefully engineered training pipelines. Progress therefore requires data-efficient synthesis, verifiable training objectives, and sustainable model-maintenance protocols.

These trade-offs imply a deployment-dependent choice. Prompt-based methods are appropriate when rapid deployment and minimal training data are the primary constraints.

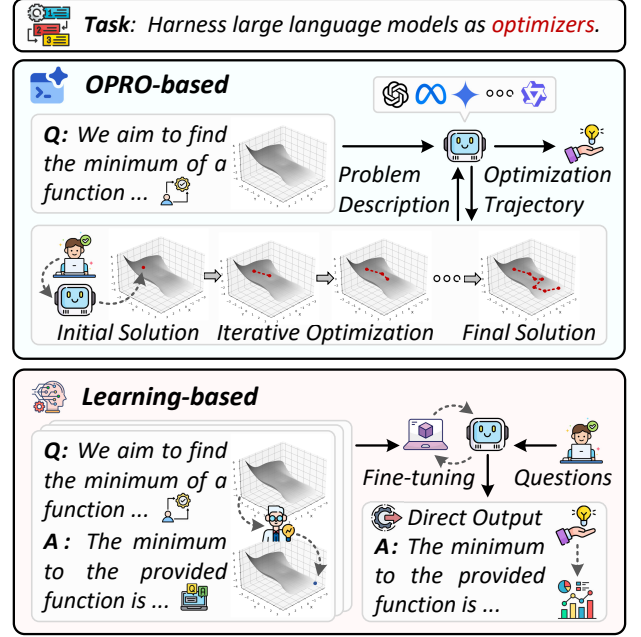


Fig. 5. Illustration of LLMs as optimizers. Most methods use iterative prompting, while a smaller group applies fine-tuning or pretraining to improve optimization performance.

Learning-based methods are preferable when reliability, privacy, and repeated use within a stable domain justify the training cost. Section V-B evaluates this distinction empirically and converts it into practitioner-oriented selection guidance.

IV. LLMs FOR OPTIMIZATION SOLVING

Optimization solving is the execution stage in which an algorithm searches for solutions to a formulated problem. Current research assigns LLMs three distinct roles in this stage: stand-alone optimizers, low-level components within optimization algorithms, and high-level systems for algorithm selection or generation. Section IV-A reviews methods that use LLMs to generate solutions directly, as illustrated in Fig. 5. Sections IV-B and IV-C then examine component-level assistance and algorithm-level orchestration, as illustrated in Figs. 6 and 7. Section IV-D compares the limitations of the three paradigms, and Supplementary Document II summarizes representative studies.

A. LLMs as Optimizers

Stand-alone LLM optimizers use in-context learning to improve candidate solutions through iterative interaction. OPRO [28] established this paradigm by expressing an optimization problem in natural language and repeatedly prompting the LLM with previous solutions and their feedback. The model infers patterns from these solution-feedback trajectories and proposes the next candidate. This mechanism is most natural for discrete or language-structured tasks, where candidates can be represented semantically and inspected by a human. Guo *et al.* [81] evaluated the paradigm on gradient descent, hill climbing, grid search, and black-box optimization. Subsequent

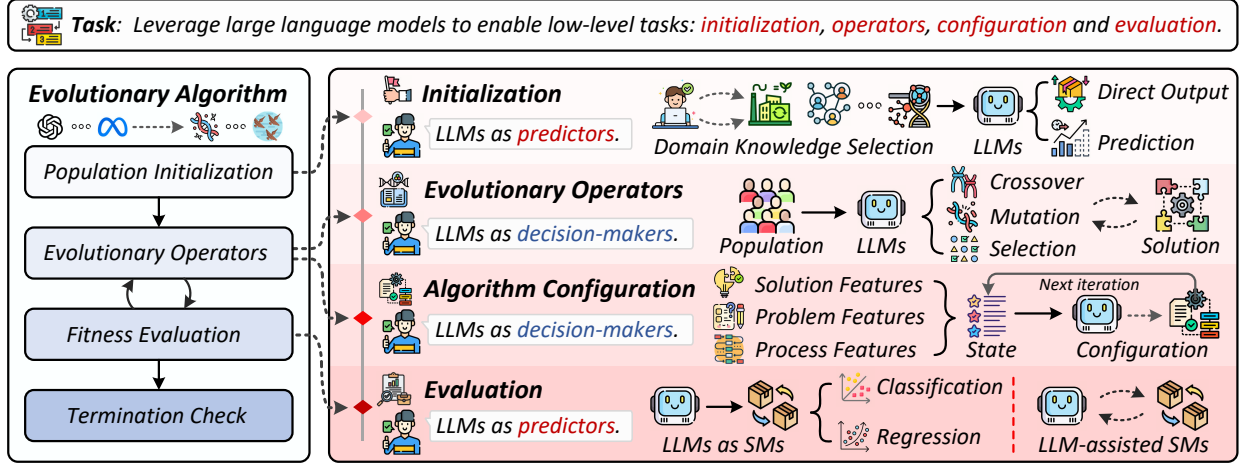


Fig. 6. Illustration of low-level LLMs for optimization algorithms. LLMs can be applied at various stages within EAs, including initialization, evolutionary operators, algorithm configuration, and fitness evaluation.

applications include structural-matrix ordering [82], wireless-network design [83], hardware-workflow parameter tuning [84], and jailbreak-attack analysis [85].

Later studies improved the information supplied during iteration rather than relying only on raw solution-feedback histories. Lange *et al.* [86] provide sorted and discretized candidates as context, which reframes candidate generation as a sequence-recognition problem. OPTO [87] supplies structured execution traces, including intermediate values and function calls, to support a backpropagation-like update process. Multimodal methods add visual structure to the prompt. Huang *et al.* [88] combine textual vehicle-routing descriptions with maps, Elhenawy *et al.* [89] combine scatter plots with multi-agent reasoning for the traveling-salesman problem, and Zhao *et al.* [90] render graphs as images so that multimodal LLMs can access topological information. These methods indicate that structured traces and visual representations can provide more useful optimization context than unstructured numerical histories.

Empirical studies nevertheless identify clear performance boundaries for stand-alone LLM optimization. Zhang *et al.* [91] show that OPRO is strongly model-dependent and that smaller models have difficulty using long iteration histories. Huang *et al.* [30] further compare LLMs on discrete and continuous black-box tasks. Their results confirm relative strengths on symbolic and language-structured problems but expose weak performance on numerical continuous optimization. The limitation becomes more pronounced when a problem lacks semantic structure, requires precise numerical calibration, has high dimensionality, or imposes a small evaluation budget. Thus, the effectiveness of this paradigm depends strongly on how naturally the optimization state can be represented in the model’s input space.

Two lines of work respond to these limitations. The first attempts to improve stand-alone optimization through training. Abgaryan *et al.* [92] fine-tune an LLM on instruction-solution pairs for single-step prediction, POM [21] pretrains an optimizer for zero-shot transfer across black-box problems, and LLOME [93] combines a two-level architecture with prefer-

ence learning to generate valid sequences under biophysical constraints. The second line changes the role of the LLM. Rather than replacing an optimization algorithm, it embeds the LLM within an established framework and assigns it a narrower decision task. This alternative motivates the low-level assistance paradigm reviewed next.

B. Low-level LLMs for Optimization Algorithms

Low-level assistance exploits the complementary strengths of population-based search and language-model reasoning [34]. EAs provide explicit mechanisms for exploration, selection, and constraint handling, while LLMs contribute semantic priors, domain knowledge, and generative decisions. Embedding an LLM within an EA therefore preserves the algorithmic search structure and restricts the LLM to tasks for which its knowledge or reasoning is useful. We organize this literature around four stages of an optimization algorithm: initialization, evolutionary operators, algorithm configuration, and evaluation.

1) **Initialization:** Initialization determines the starting distribution of the search and can strongly affect convergence and final solution quality. LLMs can inject prior knowledge by proposing candidates that are plausible before any objective evaluations are available. In neural architecture search (NAS) [94], [95], Yu *et al.* [96] use LLMs to propose architectural components, while Jawahar *et al.* [97] use an LLM as a predictor to guide initial candidate selection. In bioengineering, Teukam *et al.* [98] generate mutant libraries for enzyme design, and De *et al.* [99] generate initial portfolios for financial optimization. However, initialization through an LLM incurs inference cost and does not guarantee feasibility. Zhao *et al.* [100] show that these limitations become more severe under strict constraints and at larger problem scales.

2) **Evolutionary Operators:** Evolutionary operators determine how an EA transforms existing solutions into new candidates. Before LLMs, neural models such as attention and feed-forward networks were trained to approximate individual operators or complete algorithms [101]. Iterative-prompting

methods such as OPRO [28] suggested a training-free alternative: an LLM can generate variations after reading the problem description and selected parent solutions. This formulation allows the operator to use semantic and domain information that is difficult to encode in conventional variation rules.

Several frameworks implement this idea at different levels of the evolutionary cycle [102]. LMX [103] uses few-shot prompting for crossover and recombination of text-based genomes and generates semantically coherent offspring without additional training. LMEA [104] prompts LLMs to perform selection, crossover, and mutation and adapts the sampling temperature to balance exploration and exploitation. Other methods specialize the role further: PAIR [105] focuses on selection, LMPSO [106] adapts LLM guidance to particle swarm optimization, and LEO [107] uses LLM-generated strategies to regulate exploration and exploitation.

Operator-level assistance has also been extended to multi-objective optimization. Liu *et al.* [108] integrate zero-shot LLM prompting with MOEA/D [109] and use the observed behavior to inform the design of efficient white-box operators. Because repeated LLM calls are expensive, hybrid methods invoke the model selectively. Wang *et al.* [110] use LLMs to generate only 10% of each population, while conventional operators generate the remainder. Liu *et al.* [111] invoke an LLM only after population improvement becomes insufficient and otherwise rely on NSGA-II [112]. These studies show that selective invocation can retain useful semantic variation while controlling computational cost.

3) **Algorithm Configuration:** Algorithm configuration determines hyperparameters and operator choices that strongly influence EA performance. Conventional research distinguishes offline tuning from online control. Recent MetaBBO methods formulate online control as a Markov decision process [113] and learn policies through reinforcement learning [114]–[116]. However, these policies usually require problem-specific training and may generalize poorly outside the training distribution. LLMs offer a training-free alternative that maps observed search features to configuration decisions, although their ability to control numerical parameters remains uncertain.

Existing studies apply LLMs to both static tuning and dynamic control. Kramer *et al.* introduce LLM-based feedback loops for static parameter tuning [117] and dynamic control [118] in evolution strategies. Custode *et al.* [119] feed optimization trajectories to an OPRO-style controller for step-size adaptation and ask the model to justify each update. LAOS [120] reduces the redundancy of full trajectories by constructing a meta-prompt from features of the solution space, decision space, and search process; it then combines these features with prior optimization knowledge for adaptive operator selection. Algorithm-generation frameworks such as EoH [29] and LLaMEA [121] also tune parameters, but their primary objective is to generate algorithms rather than configure a fixed EA.

4) **Evaluation:** Evaluation is a major computational bottleneck when objective functions require expensive simulations or experiments. Surrogate-assisted optimization reduces this cost by predicting objective values or candidate quality from previously evaluated solutions. LLMs have been studied both

as direct surrogates and as managers of surrogate models. Hao *et al.* [122] formulate model-assisted selection as classification and regression and use historical observations to evaluate new candidates without additional training. LICO [123] uses LLM representations to address data scarcity in molecular optimization. At the management level, Rios *et al.* [124] use LLMs to support surrogate selection and training for engineering problems, while LLM-SAEA [125] coordinates multiple expert agents to select surrogate models and infill criteria dynamically.

LLM-based surrogates have further been extended to multitask optimization. Zhang *et al.* [126] represent task observations as token sequences and use an LLM as a meta-surrogate to transfer information across tasks. Together, these studies show that LLMs can support evaluation either by predicting candidate quality or by coordinating conventional surrogate models. However, their robustness, calibration, and scalability require further evaluation before they can replace established surrogate-learning methods.

C. High-level LLMs for Optimization Algorithms

High-level assistance assigns the LLM decisions that affect the optimization algorithm as a whole rather than individual search operations. The LLM either selects an existing algorithm for a problem instance or generates an algorithm tailored to the task. This role allows the model to reason over problem descriptions, algorithm properties, and previous performance records at a broader level. Accordingly, this section reviews two forms of high-level assistance: *algorithm selection* and *algorithm generation*.

1) **Algorithm Selection:** Algorithm selection identifies the most suitable method from a portfolio for a given problem instance. The task is necessary because algorithms exhibit different strengths across problem structures and application domains. Classical selection systems formulate it as a machine-learning problem with two stages [127]. The first stage extracts statistical or landscape features that characterize the problem and candidate algorithms. The second stage maps these features to a decision through classification [128], regression [129], or hybrid models [130]. The reliability of the final selection therefore depends on both feature quality and the selector model.

LLMs have been introduced into both stages of this workflow. AS-LLM [131] uses code understanding to extract high-dimensional representations from source code or textual descriptions. A feature-selection module retains the most relevant dimensions, separate networks encode algorithms and problems, and a similarity module produces the final selection. InstSpecHH [132] instead emphasizes the selector. It first filters candidate algorithms by Euclidean distance and then asks an LLM to compare natural-language descriptions of the problem and the remaining algorithms. Other systems select mathematical solvers inside broader modeling workflows [51], [52], but this task differs from classical portfolio-based algorithm selection and is treated separately in this survey.

2) **Algorithm Generation:** Algorithm generation constructs an optimization algorithm from problem characteristics rather

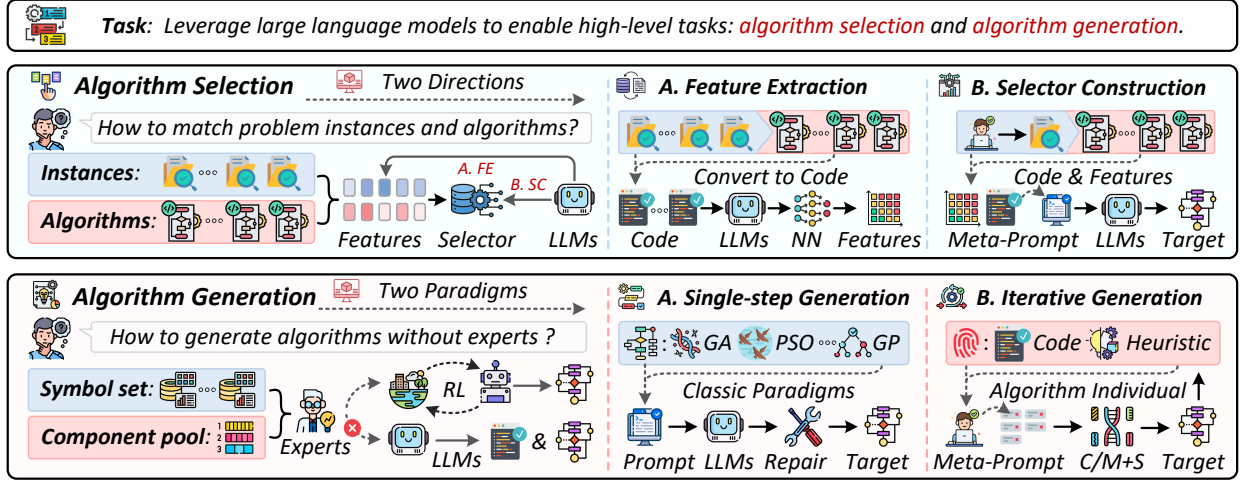


Fig. 7. Illustration of high-level LLM-assisted optimization algorithms. Algorithm selection involves two key stages: feature extraction and selector construction. Algorithm generation has evolved from single-step to iterative generation, which reflects the shift toward more adaptive and context-aware design.

than selecting from a fixed portfolio. Earlier reinforcement-learning methods reduce manual design but still depend on predefined search spaces [12]. GSF [133] searches within a generic EA template whose component pools are specified manually, while SYMBOL [19] removes the component pools but retains a manually defined symbol set. LLMs can generate algorithmic concepts and executable code without an explicit component library, which expands the design space but also makes validation more difficult.

Research has progressed from single-step generation to iterative search [131]. In early demonstrations, Pluhacek *et al.* [134] prompt GPT-4 to decompose and recombine six metaheuristics, while Zhong *et al.* [135] generate the ZSO metaheuristic with GPT-3.5 and prompt engineering. These studies establish feasibility but rely almost entirely on the prior knowledge and one-shot reasoning of the underlying model. This dependence motivates iterative frameworks that evaluate generated algorithms and feed performance information back into the generation process.

Iterative generation now forms the main line of algorithm-generation research. FunSearch [136] combines LLM generation with evolutionary selection to improve program fragments in function space. AEL [137] and EoH [29] extend this idea by co-evolving heuristic concepts and executable implementations. EoH represents a heuristic as a natural-language idea and asks the LLM to produce corresponding code. This dual representation permits search in an abstract semantic space while retaining executable evaluation, which can be more efficient than varying source code alone. Reported experiments show that EoH generates heuristics that outperform established handcrafted baselines on several combinatorial benchmarks.

This line of work has produced supporting tools and extensions. LLM4AD [138] provides a platform for LLM-assisted algorithm design, while subsequent studies characterize the rugged and multimodal fitness landscape of the search [139] and fine-tune LLMs through diversity-aware ranking, sampling, and direct preference optimization [140]. MEoH [141] extends EoH to multi-objective heuristic search, and EoH-S

[142] co-evolves complementary algorithm sets. Applications include edge-server scheduling [143], Bayesian optimization [144], and adversarial-attack design [145].

The LLaMEA series [121], [146] follows a complementary code-centered trajectory. It uses an LLM as a variation operator within a conventional evolutionary loop and refines source code according to performance measurements and runtime feedback. Unlike EoH, it does not explicitly co-evolve natural-language concepts and code. The authors analyze the resulting search through code-evolution graphs [147] and behavior-space representations [148], and they introduce BLADE [149] for standardized evaluation. LLaMEA-HPO [150] adds hyperparameter optimization to reduce iteration cost. The framework has also been applied to Bayesian optimization [151] and photonic-structure design [152].

Other iterative frameworks explore alternative search and reflection mechanisms [153]–[158]. ReEvo [159] treats the LLM as a hyper-heuristic and uses reflective evolution to revise candidate strategies. MCTS-AHD [160] applies Monte Carlo Tree Search to the heuristic design space to reduce premature convergence. HSEvo [161] combines harmony search with genetic algorithms and uses diversity measures to balance exploration and exploitation. Collectively, these methods mark a transition from isolated one-shot demonstrations to search frameworks that generate, evaluate, and revise algorithms iteratively.

D. Challenges

The three solving paradigms exhibit different strengths because they assign the LLM decisions at different structural levels. Stand-alone methods such as OPRO [28] ask the model to generate solutions directly. Low-level methods such as LMEA [104] and LMX [103] restrict the model to particular operations within an EA. High-level methods such as EoH [29] and LLaMEA [121] search over algorithm designs. This structural distinction produces three corresponding bottlenecks:

- **LLMs as Optimizers:** The main limitation is a mismatch between autoregressive sequence prediction and numer-

ical search. Stand-alone methods must encode the optimization state as a sequence and infer useful updates from solution-feedback histories. Long, unstructured histories disperse attention and can obscure important intermediate states [30], [91]. The problem becomes more severe for population-based search because the context must represent many candidates and their relations. Our BBOB experiments show that performance degrades sharply as dimensionality increases and that LLM optimizers stagnate even on the Sphere function. Thus, this paradigm is most suitable for discrete or language-structured tasks and remains unreliable for high-precision, high-dimensional continuous optimization. Future work should compress trajectories into structured state summaries or reposition the LLM as a problem-understanding module within a numerical optimizer.

- **Low-level LLMs for Optimization Algorithms:** The main limitation is the scale and interdependence of the assigned decision space. LLMs can be effective for local, discrete operations, but global tasks such as population initialization and full algorithm configuration require many coordinated decisions across combinatorial or continuous variables. Autoregressive generation provides no explicit guarantee of global feasibility or consistency. LLM controllers also tend to make conservative parameter updates, which can reduce exploration [100], [119]. Our parameter-control experiments further indicate that discrete action representations produce more effective exploration than direct continuous outputs. Low-level assistance is therefore most reliable when the decision scope is narrow and feedback is immediate. Future methods should use hierarchical decomposition to convert global decisions into constrained subproblems that an LLM can address separately.
- **High-level LLMs for Optimization Algorithms:** Algorithm generation searches a semantic design space that combines heuristic concepts with executable code, typically through EC or Monte Carlo Tree Search. This process faces two main limitations. First, it is computationally expensive because each candidate may require an LLM call, code execution, and evaluation on multiple benchmark instances. Second, hierarchical design remains difficult: the model must map a problem description to an algorithmic framework, operators, and parameter settings while preserving executability and constraints. Current methods therefore tend to recombine established strategies more often than they produce fundamentally new algorithmic principles. Future work should develop surrogate evaluators for lower-cost search and connect LLMs with formal reasoning and verification modules for hierarchical, constraint-aware synthesis.

These limitations support a paradigm-aware deployment principle. Classical EAs remain the default for numerical and high-dimensional optimization. LLMs are more useful when the task contains substantial language or semantic structure, or when the model is assigned a narrow decision role within an established algorithm. Section V-B evaluates this principle

empirically and translates it into method-selection guidance.

V. EMPIRICAL STUDY AND GUIDANCE

To ground the preceding taxonomy empirically, this section systematizes the benchmark landscape and compares representative methods across the modeling-to-solving workflow. Section V-A reviews benchmarks and evaluation protocols for both modeling and solving. Section V-B then combines collected results with controlled experiments on optimization modeling, stand-alone LLM optimization, low-level assistance, and high-level algorithm generation. Figs. 8, 10, 11, and 9 summarize the corresponding evidence.

A. Benchmarks and Evaluation Protocols

Benchmark development has accompanied the rapid growth of LLM-based optimization methods. A useful benchmark must satisfy two requirements: it should contain sufficiently diverse problems to expose method limitations, and it should support centralized evaluation under comparable protocols. We review the benchmark landscape separately for optimization modeling and optimization solving.

Optimization-modeling benchmarks have developed through three stages. The first was competition-driven and centered on NL4Opt [40]. The second expanded coverage through manually curated datasets, including MAMO [62], NLP4LP [52], NL2OPT [54], and ComplexOR [50]. The third introduced scalable synthetic generation, as represented by IndustryOR [27], OptiBench [72], OptMath [73], DP-Bench [74], and DCP-Bench [162]. NL4Opt, MAMO, NLP4LP, ComplexOR, IndustryOR, OptiBench, and OptMath are among the most widely used datasets in recent studies [80], [163], [164]. Supplementary Document III provides further dataset details.

Evaluation protocols fall into objective-wise and model-wise families. Objective-wise evaluation executes the generated formulation with a solver and compares the resulting objective value with the ground truth. CoE [50] introduced this test-driven protocol, and pass@1 has become the most common metric. However, a correct objective value does not guarantee that the generated model is structurally correct. Model-wise evaluation therefore compares formulations directly. NL4Opt converts models into coefficient matrices, while later methods use graph-edit distance [165] or modified graph-isomorphism tests with theoretical guarantees [63]. These measures provide finer-grained correctness scores, but they require reference models that are costly to construct. Consequently, pass@1 remains dominant because it is easy to compute and directly reflects downstream solver behavior.

Benchmarks for optimization solving are less evenly distributed across the three paradigms. Most existing suites focus on stand-alone LLM optimizers or high-level algorithm generation, while low-level assistance lacks a common benchmark. For stand-alone optimization, NLGraph [166], PPLN [167], and GraphArena [168] primarily evaluate one-shot solutions on graph or language-structured tasks. Opt-Bench [169] extends this setting to iterative solving, in which the model revises candidates over multiple rounds. Algorithm-generation

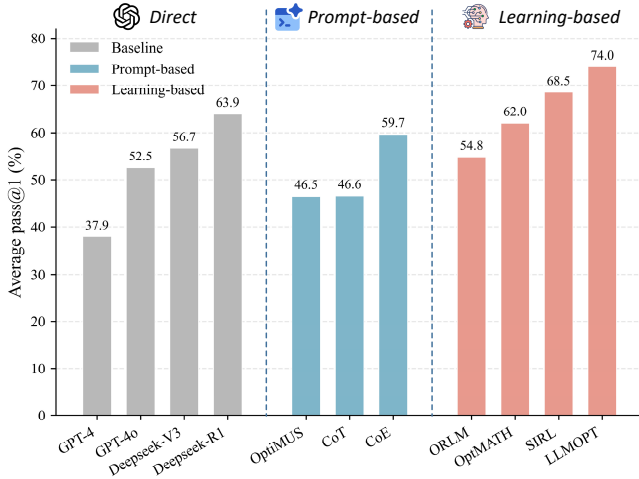


Fig. 8. Average pass@1 of representative methods from each category across all eight optimization modeling benchmarks. Methods are grouped as baseline, prompt-based, and learning-based approaches and are sorted within each group.

benchmarks have developed more rapidly and include CO-Bench [170], FrontierCO [171], HeuriGYM [172], ALE-Bench [173], and the LLM4AD platform [138]. Supplementary Document III summarizes these resources.

Evaluation of generated algorithms commonly combines three quantities. The first is optimization quality relative to a best-known solution or an expert-tuned baseline. The second is validity, which includes code executability and solution feasibility. The third is cross-problem performance, which is often aggregated through normalized scores or rankings. Although individual benchmarks implement these quantities differently, no single protocol has yet become standard.

B. Baseline Comparison and Guidance

We compare the four parts of the taxonomy using either curated results from common benchmarks or controlled experiments against established baselines. For each part, we report the empirical pattern, identify its practical implication, and state the corresponding research opportunity.

For optimization modeling, we compare methods that report results on all eight selected benchmarks. The baseline group contains GPT-4, GPT-4o, DeepSeek-V3, and DeepSeek-R1. The prompt-based group contains OptiMUS [52], chain-of-thought prompting [174], and CoE [50], all implemented with GPT-4o. The learning-based group contains ORLM [27], OptMATH [73], SIRL [78], and LLMOPT [77]. Fig. 8 reports mean pass@1 across the eight benchmarks. To maintain comparability, we include only methods evaluated on every benchmark and collect results from the original papers, benchmark reports, and recent comparative studies such as [163]. Supplementary Document IV provides the data-collection criteria, per-dataset results, and additional baselines.

The results favor learning-based methods in the current comparison. Despite using open-source backbones with 8–32 billion parameters, LLMOPT and SIRL achieve average pass@1 scores of 74.0 and 68.5, respectively, compared with

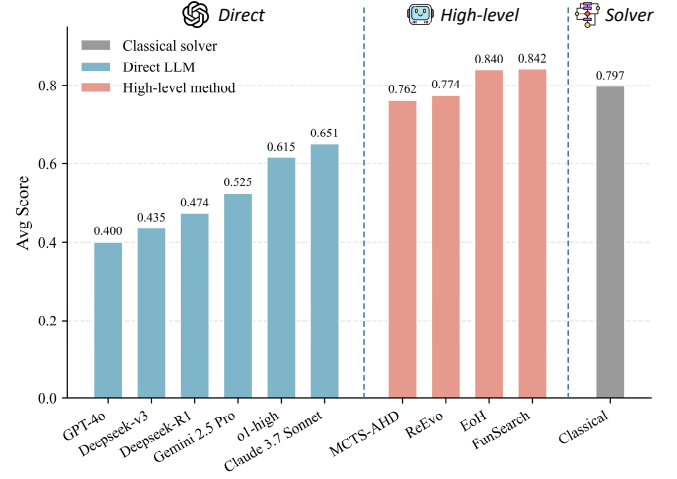


Fig. 9. Average normalized objective scores on CO-Bench for three groups: a well-tuned classical solver, direct LLM generation, and high-level agentic frameworks. Scores are averaged across all problems, with higher values indicating better performance and 1.0 matching the best-known solution.

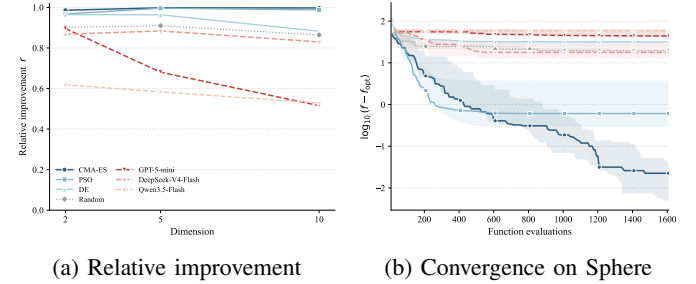


Fig. 10. Performance comparison between LLM-as-optimizer and classical EAs on BBOB benchmarks. (a) Relative improvement r averaged over four functions and five seeds; the results characterize how LLM-optimizer performance changes with dimensionality. (b) Median convergence trajectories on the 10-dimensional Sphere function; the trajectories reveal the early stagnation of LLM optimizers.

63.9 for DeepSeek-R1. Prompt-based performance is more variable: CoE reaches 59.7, whereas chain-of-thought prompting and OptiMUS remain near 46.5. For practitioners who need a model without additional training, a strong closed-source LLM or an available specialized model such as LLMOPT is the most direct option. When direct prompting is insufficient, search-augmented workflows such as CoE can improve accuracy without model training, but they increase API cost. For researchers, the central question is whether specialized fine-tuning can maintain its advantage as general-purpose reasoning models improve, particularly when the open-source backbone limits the final performance.

To evaluate LLMs as numerical optimizers, we compare OPRO-style iterative optimization [28] with classical methods on four BBOB functions [175] across multiple dimensions and random seeds. The classical baselines are random search, differential evolution (DE) [176], particle swarm optimization (PSO) [177], and CMA-ES [178]. The LLM optimizers use GPT-5-mini, DeepSeek-V4, and Qwen3.5 as backbones, and relative improvement r is the primary metric. Supplementary Document V reports the complete setup and results.

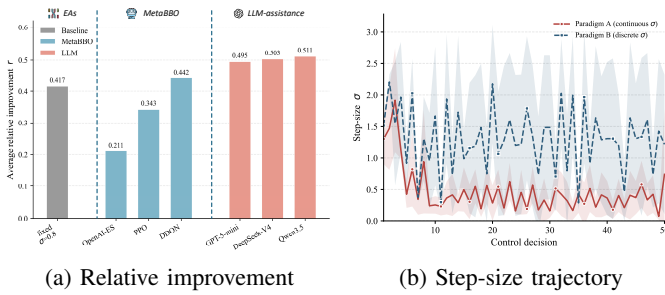


Fig. 11. Evaluation of LLM-driven step-size control in (1+1)-ES. (a) Average relative improvement r across baseline, MetaBBO, and LLM controllers. (b) Step-size trajectory of Qwen3.5 on the Rastrigin function; the trajectory contrasts the exploitative behavior of continuous output with the exploratory behavior of discrete output.

Fig. 10 shows the mean relative improvement and the median convergence trajectory on the 10-dimensional Sphere function. The classical methods retain strong performance as dimensionality increases; CMA-ES and PSO both achieve $r > 0.98$ at dimension 10. The LLM optimizers perform worse and degrade with dimensionality. For example, GPT-5-mini decreases from $r = 0.896$ at dimension 2 to $r = 0.516$ at dimension 10. The Sphere trajectories further show early stagnation. These results indicate that current iterative LLM optimization does not match mature EAs on numerical continuous problems. Practitioners should therefore retain classical EAs as the default for this setting. A more promising research direction is to convert raw trajectories into structured state summaries that preserve optimization-relevant information without requiring the model to reason directly over long vectors of floating-point values.

We use parameter control in a (1+1)-ES as a representative low-level task and compare LLM controllers with learning-based MetaBBO controllers. DDQN [179], PPO [12], OpenAI-ES [180], and the LLM controllers [120] receive the same state observation. The LLM group again uses GPT-5-mini, DeepSeek-V4, and Qwen3.5, with relative improvement r as the evaluation metric. We also compare continuous-output control [119] with discrete-action control [120] on Rastrigin to isolate the effect of the output representation. Supplementary Document VI provides the full setup.

As shown in Fig. 11, all three LLM controllers outperform the fixed baseline and the trained MetaBBO controllers in this experiment. This result contrasts with the weak performance of stand-alone LLM optimization and suggests that an LLM can be effective when assigned a small, well-defined decision problem. The representation comparison further shows that discrete actions produce discernible exploration and exploitation, whereas direct continuous outputs remain concentrated within a narrow range. For practitioners, LLM control is therefore a viable option for limited dynamic tuning, particularly when the action space can be discretized. For researchers, the next step is to evaluate control at multiple granularities and to determine how population or variable decomposition can create decision units that remain compatible with LLM reasoning.

For high-level assistance, we focus on algorithm generation because it is the most active part of this paradigm.

Benchmark metrics remain heterogeneous, so we use CO-Bench [170], which compares direct LLM generation and agentic frameworks with a tuned classical solver under an average normalized objective score. Supplementary Document VII relates these results to other studies and provides an error analysis.

Fig. 9 shows a clear distinction between one-shot generation and iterative agentic search. The classical solver achieves an average score of 0.797, while the strongest direct-generation model, Claude 3.7 Sonnet, reaches 0.651. Reasoning-oriented models consistently outperform their non-reasoning counterparts, but one-shot generation remains below the classical baseline. In contrast, FunSearch [136] and EoH [29] reach 0.842 and 0.840, respectively, while ReEvo [159] and MCTS-AHD [160] achieve 0.774 and 0.762. The strongest agentic methods outperform the classical solver on more than half of the test instances.

This advantage is qualified by feasibility and novelty limitations. Even the best agents have lower valid-solution rates than the classical solver, which reduces per-instance reliability. Moreover, generated algorithms commonly recombine established techniques such as vectorization, local search, and simulated annealing rather than introduce new algorithmic principles. For practitioners, agentic generation is competitive when iterative development and evaluation are affordable, whereas classical solvers remain preferable when feasibility guarantees are essential. For researchers, the main opportunities are constraint-aware generation, lower-cost search, sustained improvement beyond fixed iteration budgets, and algorithmic novelty rather than more elaborate recombination of known components.

The empirical evidence supports a problem-driven selection rule rather than a universally best paradigm. The first decision axis is problem structure. Language-structured tasks, including mathematical formulation and combinatorial algorithm design, can benefit substantially from LLMs, whereas numerical and high-dimensional continuous problems remain better served by classical EAs. The second axis is decision granularity. LLMs are more effective for narrow and dynamic decisions, such as discrete parameter control, than for global or population-scale search. The third axis is deployment constraints. When feasibility, per-instance reliability, or data privacy is critical, classical solvers or locally deployed fine-tuned models are preferable to closed-source APIs. Available training resources then determine whether prompt-based deployment or fine-tuning is appropriate. Fig. 12 consolidates these considerations into a method-selection flowchart.

VI. APPLICATIONS

LLM-assisted optimization has been applied beyond benchmark numerical and combinatorial problems to tasks in computer science, the natural sciences, and engineering. These applications differ in how they use the model: some rely on language-based problem formulation, some embed LLMs as operators or surrogates, and others use them to generate complete algorithms. Section VI-A reviews machine-learning and security applications. Section VI-B covers chemistry,

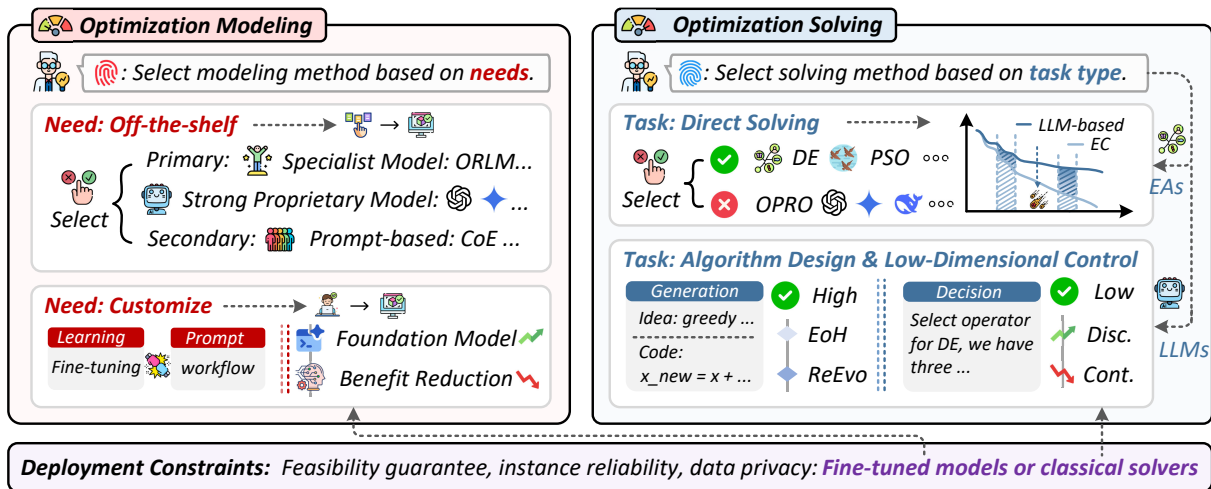


Fig. 12. Method-selection flowchart that maps problem traits and deployment requirements to recommended LLM-for-optimization paradigms across the modeling route and the three solving paradigms.

biology, and physics. Section VI-C examines engineering and industrial systems.

A. Computer Science

In computer science, neural architecture search (NAS) [181] and security optimization [145] are two representative application areas, while data augmentation, code optimization, and other tasks further demonstrate the breadth of the field [182]–[185]. NAS research mainly uses low-level assistance rather than stand-alone LLM optimization [186]. Chen *et al.* [181] use LLMs as crossover and mutation operators within an EA [187]; few-shot prompts generate candidate architectures, and mixed sampling temperatures increase diversity. This design has been extended to quality-diversity search with dual archives [188], role-based prompt diversification [189], and graph neural architecture search [190]. LLMs also support initialization through pretraining [96] or few-shot prediction [97].

Security applications use LLMs both as direct optimizers and as search operators. Jiang *et al.* [85] generate jailbreak suffixes through iterative self-reflection. Other methods embed LLMs as heuristic operators for jailbreak-prompt optimization and improve efficiency or transferability [191], [192]. AutoDA [145] instead optimizes the generation function that produces adversarial examples, which allows attack strategies to evolve over repeated evaluations.

Related computer-science applications reformulate design artifacts as language-editable objects. Wang *et al.* [182] represent data-augmentation policies as natural-language instructions and apply crossover and mutation to adapt them to long-tailed data. Li *et al.* [183] iteratively generate and optimize CUDA kernels and use execution performance as feedback without manual code revision.

B. Natural Sciences

Natural-science problems in chemistry [193], biology [194], and physics [195] often combine high-dimensional search

spaces, expensive evaluations, and domain knowledge that is difficult to encode explicitly. LLMs are therefore used primarily to inject scientific priors, generate structured candidates, or coordinate expensive search rather than to replace numerical optimization entirely.

Molecular discovery is the main chemistry application of LLM-assisted optimization [193]. Most methods embed chemically informed LLMs within EAs. Wang *et al.* [196] use an LLM for molecular crossover and mutation. Guevorguian *et al.* [197] replace genetic operators with fine-tuned LLMs and use a predictive model to reduce stagnation; the approach is later extended to multi-objective molecular discovery [198]. At the evaluation level, LICO [123] uses LLM representations as surrogates when molecular data are scarce.

Biological applications also favor component-level integration. LLM-GA [194] uses an LLM to initialize mutant libraries for enzyme design. Tran *et al.* [199] and Wang *et al.* [200] use LLMs as mutation or crossover operators for protein design. Chen *et al.* [93], [201] develop a two-level optimizer that iteratively modifies biological sequences under complex constraints. ProLLaMA [202] spans both modeling and solving by mapping natural-language descriptions to protein-function predictions and then optimizing the corresponding sequences.

Physics applications span modeling and solving across fluid dynamics [195], [203], semiconductor systems [204], and statistical physics [205], [206]. At the modeling level, Du *et al.* [205] use GPT-3.5 to derive partial differential equations from data, Zhang *et al.* [195] apply LLMs to turbulence-closure modeling, and Ma *et al.* [207] connect modeling with solving in a unified workflow. At the solving level, Li *et al.* [204] use LLMs to generate and debug code for laser-parameter optimization, while Zhang *et al.* [203] use LLMs to propose parameter combinations for fluid-dynamics simulations.

C. Engineering and Industry

Engineering and industrial applications concentrate on wireless communications [83], industrial design [152], and edge-

computing or scheduling problems [143]. These domains combine structured engineering knowledge with expensive simulation or deployment constraints, which creates opportunities for LLMs at both the modeling and solving stages.

Wireless-network research covers the complete workflow from formulation to search. For modeling, LLM-OptiRA [208] identifies nonconvex components in resource-allocation problems and reformulates them into solvable forms. Wen *et al.* [209] use retrieval-augmented generation to incorporate external expert knowledge into the formulation process. For solving, Qiu *et al.* [83] use LLMs directly for combinatorial tasks such as access-point placement, and related studies address resource allocation [210], power control [211], and multi-UAV deployment [212]. Low-level methods embed LLMs in multi-objective optimization for integrated sensing and communication in UAV networks [213]. The LHS framework [214] combines heuristic recommendation, informed initialization, and iterative refinement.

Industrial-design applications use LLMs for configuration, direct search, and algorithm generation. Ghose *et al.* [84] develop an agent that tunes chip-design parameters for performance, power, and area. Jiang *et al.* [82] use LLMs to optimize design-structure matrices that encode dependencies within engineering systems. Yin *et al.* [152] apply LLaMEA [121] to generate algorithms for photonic-structure design. Beyond design, Yatong *et al.* [143] apply LLM-generated heuristics to task scheduling in edge-server environments.

VII. VISION FOR THE FIELD

The reviewed literature reveals substantial methodological diversity, but it also exposes three structural gaps that limit further progress. First, optimization modeling and solving remain weakly connected. Second, most methods are designed offline and adapt only minimally during search. Third, existing systems usually assign an LLM a single isolated role rather than organizing multiple agents, solvers, tools, and human experts into a persistent optimization ecosystem. These gaps motivate three corresponding research directions: integrated modeling and solving, dynamic self-adaptation, and agentic coordination.

A. Bridging Modeling and Solving

Optimization modeling and solving have largely developed as separate research lines. OptiMUS [51], [52] connects natural-language modeling with program generation through an agent-based conductor, while ORLM [27] generates mathematical formulations and solver code through a fine-tuned model. However, both systems ultimately rely on conventional external solvers. The LLM therefore remains concentrated at the front end and cannot exploit advances in adaptive configuration, algorithm selection, or algorithm generation during the solving process.

Bridging the two stages requires both sequential integration and reciprocal interaction. The first direction is an *end-to-end LLM-driven workflow* that links problem understanding, formulation, algorithm design, execution, and evaluation within one closed loop. Such a system should not merely call a fixed

external solver; it should select, configure, or generate solving strategies according to the formulated problem and observed search behavior. The second direction is parallel co-evolution of the model and the algorithm. Model-derived structures and constraint signatures can condition algorithm design, while search trajectories and failure modes can expose missing constraints or modeling errors and trigger reformulation. This reciprocal feedback would turn modeling and solving into mutually corrective processes rather than a one-way pipeline.

B. Transitioning from Static to Dynamic Methods

Most current LLM-based optimization methods are *static*: their prompts, algorithm structures, and interaction protocols are specified before the search and change little in response to online evidence. Reinforcement-learning research provides a contrasting model through dynamic configuration [179], [215], dynamic selection [216], and dynamic generation [19]. These methods adapt decisions to the current optimization state. Dynamic selection schedules algorithms according to their observed strengths, while dynamic generation constructs operators for changing search conditions.

Dynamic selection and generation remain underexplored in LLM-based optimization. Existing adaptive examples are mostly local, including interactive modeling [57] and parameter control [119], [120]. A broader objective is the *self-evolution of algorithms*, in which an LLM revises representations, operators, parameters, and coordination strategies according to search feedback. Such systems would retain evaluated experience, reuse successful components, and alter their own search logic as the problem or environment changes. The central technical challenge is to support continual adaptation without uncontrolled drift, loss of feasibility, or repeated rediscovery of previously learned strategies.

C. Toward an Agentic Ecosystem for Optimization

LLMs should ultimately be viewed not as isolated optimization engines but as agents within a larger optimization ecosystem. In such a system, agents can collaborate with human experts, mathematical solvers, simulators, and domain tools. They can capture tacit knowledge, such as driver-routing practices [217] or scheduling constraints [218], and support trustworthy interaction through infeasibility diagnosis [219] and decision explanation [220]–[223].

Current multi-agent frameworks commonly rely on a centralized conductor [50], [57], [185], [224], which can become a bottleneck as the number and diversity of agents increase. EvoGit [225] provides an alternative coordination model based on a shared phylogenetic graph and asynchronous, reward-free interaction. Adapting such structure-driven coordination to optimization could allow agents to develop and compare algorithms through a shared partial-order representation without requiring a single controller to determine every interaction.

An agentic optimization ecosystem must also support adaptation and knowledge transfer. Real-world objectives and constraints change over time [57], so the system should detect distribution shift through performance degradation, constraint

violations, or changes in user requirements. It should then revise its modeling and solving strategies online. This process would extend interactive formulation toward continual task-aware optimization [226].

Cross-domain transfer is equally important. Although LLMs contain broad pretrained knowledge, most current optimization systems remain specialized to one domain or problem family. Unified representations of optimization trajectories and modular skill libraries [227] could allow agents to retrieve, compose, and adapt experience across heterogeneous tasks. The resulting ecosystem would treat algorithms, prompts, models, and domain procedures as reusable and evolvable components rather than isolated solutions.

VIII. CONCLUSION

This survey organized LLMs for optimization through a modeling-to-solving perspective. For optimization modeling, we reviewed prompt-based and learning-based methods that translate natural-language descriptions into formal variables, objectives, constraints, and solver code. For optimization solving, we distinguished three roles: stand-alone LLM optimizers, low-level LLM components embedded in established algorithms, and high-level LLM systems for algorithm selection and generation. This taxonomy clarifies how the effectiveness of an LLM depends on the structural level and decision scope assigned to it.

The benchmark analysis and baseline comparisons further identify consistent deployment boundaries. Classical EAs remain the most reliable choice for numerical and high-dimensional continuous optimization. LLMs provide greater value when the task contains substantial language or semantic structure, when they generate or select algorithms at a high level, or when they make narrow component-level decisions such as discrete parameter control. Prompt-based modeling offers rapid deployment, while fine-tuned models can improve reliability when sufficient data and maintenance resources are available.

Future progress requires systems that connect modeling with solving, adapt their algorithms during search, and coordinate multiple agents, tools, solvers, and human experts. Such systems should support reciprocal feedback between formulation and algorithm design, continual adaptation without loss of reliability, and transfer of reusable optimization knowledge across domains. By identifying these requirements, this survey provides both a structured account of the current field and a basis for developing dynamic, self-evolving, and agentic optimization systems.

REFERENCES

- [1] T. Chai, Y. Jin, and S. Bernhard, "Evolutionary complex engineering optimization: opportunities and challenges," *IEEE Computational Intelligence Magazine*, vol. 8, no. 3, pp. 12–15, 2013.
- [2] A. Mahor, V. Prasad, and S. Rangnekar, "Economic dispatch using particle swarm optimization: A review," *Renewable and Sustainable Energy Reviews*, vol. 13, no. 8, pp. 2134–2141, 2009.
- [3] K. Terayama, M. Sumita, R. Tamura, and K. Tsuda, "Black-box optimization for automated discovery," *Accounts of Chemical Research*, vol. 54, no. 6, pp. 1334–1346, 2021.
- [4] Z.-G. Chen, Z.-H. Zhan, S. Kwong, and J. Zhang, "Evolutionary computation for intelligent transportation in smart cities: A survey," *IEEE Computational Intelligence Magazine*, vol. 17, no. 2, pp. 83–102, 2022.
- [5] E. Cambria, B. White, T. Durrani, and N. Howard, "Computational intelligence for natural language processing," *IEEE Computational Intelligence Magazine*, vol. 9, no. 1, pp. 19–63, 2014.
- [6] P. Festa, "A brief introduction to exact, approximation, and heuristic algorithms for solving hard combinatorial optimization problems," in *Proceedings of the International Conference on Transparent Optical Networks*, 2014, pp. 1–20.
- [7] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 2002.
- [8] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, "Paramils: An automatic algorithm configuration framework," *Journal of Artificial Intelligence Research*, vol. 36, pp. 267–306, 2009.
- [9] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evolutionary Computation*, vol. 27, no. 1, pp. 3–45, 2019.
- [10] H. Müller-Merbach, "Heuristics and their design: A survey," *European Journal of Operational Research*, vol. 8, no. 1, pp. 1–23, 1981.
- [11] J. Zhang, Z. Zhan, Y. Lin, N. Chen, Y. Gong, J. Zhong, H. S. Chung, Y. Li, and Y. Shi, "Evolutionary computation meets machine learning: A survey," *IEEE Computational Intelligence Magazine*, vol. 6, no. 4, pp. 68–75, 2011.
- [12] Z. Ma, H. Guo, Y. Gong, J. Zhang, and K. C. Tan, "Toward automated algorithm design: A survey and practical guide to meta-black-box optimization," *IEEE Transactions on Evolutionary Computation*, 2025.
- [13] T. N. Mundhenk, M. Landajuela, R. Glatt, C. P. Santiago, D. M. Faissol, and B. K. Petersen, "Symbolic regression via neural-guided genetic programming population seeding," *arXiv preprint arXiv:2111.00053*, 2021.
- [14] Y. Wang, T. Zhang, Y. Chang, X. Wang, B. Liang, and B. Yuan, "A surrogate-assisted controller for expensive evolutionary reinforcement learning," *Information Sciences*, vol. 616, pp. 539–557, 2022.
- [15] J. E. Pettinger and R. M. Everson, "Controlling genetic algorithms with reinforcement learning," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2002, pp. 692–692.
- [16] A. Eiben, M. Horvath, W. Kowalczyk, and M. C. Schut, "Reinforcement learning for online control of evolutionary algorithms," in *Proceedings of the International Workshop on Engineering Self-Organising Applications*, 2006, pp. 151–160.
- [17] Y. Chen, M. W. Hoffman, S. G. Colmenarejo, M. Denil, T. P. Lillicrap, M. Botvinick, and N. Freitas, "Learning to learn without gradient descent by gradient descent," in *Proceedings of the International Conference on Machine Learning*, 2017, pp. 748–756.
- [18] M. G. Lagoudakis and M. L. Littman, "Algorithm selection using reinforcement learning," in *Proceedings of the International Conference on Machine Learning*, 2000, pp. 511–518.
- [19] J. Chen, Z. Ma, H. Guo, Y. Ma, J. Zhang, and Y. Gong, "Symbol: Generating flexible black-box optimizers through symbolic equation learning," *arXiv preprint arXiv:2402.02355*, 2024.
- [20] N. Mazaykina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Computers & Operations Research*, vol. 134, p. 105400, 2021.
- [21] X. Li, K. Wu, Y. B. Li, X. Zhang, H. Wang, and J. Liu, "Pretrained optimization model for zero-shot black box optimization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 14 283–14 324, 2024.
- [22] M. Han, X. Li, K. Wu, X. Zhang, and H. Wang, "Enhancing zero-shot black-box optimization via pretrained models with efficient population modeling, interaction, and stable gradient approximation," *Advances in Neural Information Processing Systems*, vol. 38, pp. 33 238–33 274, 2026.
- [23] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J. Nie, and J. Wen, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
- [24] R. Patil and V. Gudivada, "A review of current trends, techniques, and challenges in large language models (llms)," *Applied Sciences*, vol. 14, no. 5, p. 2074, 2024.
- [25] F. Mi, Y. Li, Y. Zeng, J. Zhou, Y. Wang, C. Xu, L. Shang, X. Jiang, S. Zhao, and Q. Liu, "Pangu-bot: efficient generative dialogue pre-training from pre-trained language model," *arXiv preprint arXiv:2203.17090*, 2022.

- [26] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [27] C. Huang, Z. Tang, S. Hu, R. Jiang, X. Zheng, D. Ge, B. Wang, and Z. Wang, "Orlm: A customizable framework in training large models for automated optimization modeling," *Operations Research*, 2025.
- [28] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *Proceedings of the International Conference on Learning Representations*, 2023, pp. 1–10.
- [29] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," *arXiv preprint arXiv:2401.02051*, 2024.
- [30] B. Huang, X. Wu, Y. Zhou, J. Wu, L. Feng, R. Cheng, and K. C. Tan, "Exploring the true potential: Evaluating the black-box optimization capability of large language models," *arXiv preprint arXiv:2404.06290*, 2024.
- [31] X. Wu, S. Wu, J. Wu, L. Feng, and K. C. Tan, "Evolutionary computation in the era of large language model: survey and roadmap," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 2, pp. 534–554, 2025.
- [32] S. Huang, K. Yang, S. Qi, and R. Wang, "When large language model meets optimization," *Swarm and Evolutionary Computation*, vol. 90, p. 101663, 2024.
- [33] H. Yu and J. Liu, "Deep insights into automated optimization with large language models and evolutionary algorithms," *arXiv preprint arXiv:2410.20848*, 2024.
- [34] W. Chao, J. Zhao, L. Jiao, L. Li, F. Liu, and S. Yang, "When large language models meet evolutionary algorithms," *arXiv preprint arXiv:2401.10510*, 2024.
- [35] F. Liu, Y. Yao, P. Guo, Z. Yang, Z. Zhao, X. Lin, X. Tong, M. Yuan, Z. Lu, Z. Wang, and Q. Zhang, "A systematic survey on large language models for algorithm design," *arXiv preprint arXiv:2410.14716*, 2024.
- [36] T. P. Pawlak and K. Krawiec, "Automatic synthesis of constraints from examples using mixed integer linear programming," *European Journal of Operational Research*, vol. 261, no. 3, pp. 1141–1157, 2017.
- [37] T. P. Pawlak and M. O'Neill, "Grammatical evolution for constraint synthesis for mixed-integer linear programming," *Swarm and Evolutionary Computation*, vol. 64, p. 100896, 2021.
- [38] Q. Li, L. Zhang, and V. Mak-Hau, "Synthesizing mixed-integer linear programming models from natural language descriptions," *arXiv preprint arXiv:2311.15271*, 2023.
- [39] R. Ramamonjison, H. Li, T. T. Yu, S. He, V. Rengan, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang, "Augmenting operations research with auto-formulation of optimization models from problem descriptions," *arXiv preprint arXiv:2209.15565*, 2022.
- [40] R. Ramamonjison, T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabadeh, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang, "Nl4opt competition: Formulating optimization problems based on their natural language descriptions," in *Proceedings of the Neural Information Processing Systems Competition Track*, 2023, pp. 189–203.
- [41] K. Wang, Z. Chen, and J. Zheng, "Opd@ nl4opt: An ensemble approach for the ner task of the optimization problem," *arXiv preprint arXiv:2301.02459*, 2023.
- [42] X. Doan, "Vtcc-nlp at nl4opt competition subtask 1: An ensemble pre-trained language models for named entity recognition," *arXiv preprint arXiv:2212.07219*, 2022.
- [43] N. Gangwar and N. Kani, "Highlighting named entities in input for auto-formulation of optimization problems," in *Proceedings of the International Conference on Intelligent Computer Mathematics*, 2023, pp. 130–141.
- [44] Y. Ning, J. Liu, L. Qin, T. Xiao, S. Xue, Z. Huang, Q. Liu, E. Chen, and J. Wu, "A novel approach for auto-formulation of optimization problems," *arXiv preprint arXiv:2302.04643*, 2023.
- [45] B. Almonacid, "Towards an automatic optimisation model generator assisted with generative pre-trained transformer," *arXiv preprint arXiv:2305.05811*, 2023.
- [46] D. Tsouros, H. Verhaeghe, S. Kadioğlu, and T. Guns, "Holy grail 2.0: From natural language to constraint models," *arXiv preprint arXiv:2308.01589*, 2023.
- [47] M. Jin, B. Sel, F. Hardeep, and W. Yin, "Democratizing energy management with llm-assisted optimization autoformalism," in *Proceedings of the International Conference on Communications, Control, and Computing Technologies for Smart Grids*, 2024, pp. 258–263.
- [48] T. de la Rosa, S. Gopalakrishnan, A. Pozanco, Z. Zeng, and D. Borrajo, "Trip-pal: Travel planning with guarantees by combining large language models and automated planners," *arXiv preprint arXiv:2406.10196*, 2024.
- [49] K. Liang, Y. Lu, J. Mao, S. Sun, C. Yang, C. Zeng, X. Jin, H. Qin, R. Zhu, and C.-P. Teo, "Llm for large-scale optimization model auto-formulation: A lightweight few-shot learning approach," 2025.
- [50] Z. Xiao, D. Zhang, Y. Wu, L. Xu, Y. J. Wang, X. Han, X. Fu, T. Zhong, J. Zeng, and M. Song, "Chain-of-experts: When llms meet complex operations research problems," in *Proceedings of the International Conference on Learning Representations*, 2023.
- [51] A. AhmadiTeshnizi, W. Gao, and M. Udell, "Optimus: Scalable optimization modeling with (mi) lp solvers and large language models," *arXiv preprint arXiv:2402.10172*, 2024.
- [52] A. AhmadiTeshnizi, W. Gao, H. Brunborg, S. Talaei, C. Lawless, and M. Udell, "Optimus-0.3: Using large language models to model and solve optimization problems at scale," *arXiv preprint arXiv:2407.19633*, 2024.
- [53] Z. Wang, B. Chen, Y. Huang, Q. Cao, M. He, J. Fan, and X. Liang, "Ormind: A cognitive-inspired end-to-end reasoning framework for operations research," *arXiv preprint arXiv:2506.01326*, 2025.
- [54] M. Mostajabadeh, T. T. Yu, R. Ramamonjison, G. Carenini, Z. Zhou, and Y. Zhang, "Optimization modeling and verification from problem specifications using a multi-agent multi-stage llm framework," *Infors: Information Systems and Operational Research*, vol. 62, no. 4, pp. 599–617, 2024.
- [55] A. Talebi, "Large language model-based automatic formulation for stochastic optimization models," *arXiv preprint arXiv:2508.17200*, 2025.
- [56] J. Zhang, W. Wang, S. Guo, L. Wang, F. Lin, C. Yang, and W. Yin, "Solving general natural-language-description optimization problems with large language models," *arXiv preprint arXiv:2407.07924*, 2024.
- [57] C. Lawless, J. Schoeffer, L. Le, K. Rowan, S. Sen, C. St. Hill, J. Suh, and B. Sarrafzadeh, "I want it that way: Enabling interactive decision support using large language models and constraint programming," *ACM Transactions on Interactive Intelligent Systems*, vol. 14, no. 3, pp. 1–33, 2024.
- [58] L. Gomez Tobon and E. Law, "Values in the loop: Designing interactive optimization with conversational feedback," in *Proceedings of the ACM Conference on Conversational User Interfaces*, 2025, pp. 1–5.
- [59] H. Deng, B. Zheng, Y. Jiang, and T. H. Tran, "Cafa: Coding as auto-formulation can boost large language models in solving linear programming problem," in *Proceedings of the Workshop on Mathematical Reasoning and Artificial Intelligence at the Conference on Neural Information Processing Systems*, 2024.
- [60] N. Astorga, T. Liu, Y. Xiao, and M. van der Schaar, "Autoformulation of mathematical optimization models using llms," *arXiv preprint arXiv:2411.01679*, 2024.
- [61] J. Li, R. Wickman, S. Bhatnagar, R. K. Maity, and A. Mukherjee, "Abstract operations research modeling using natural language inputs," *Information*, vol. 16, no. 2, p. 128, 2025.
- [62] X. Huang, Q. Shen, Y. Hu, A. Gao, and B. Wang, "Llms for mathematical modeling: Towards bridging the gap between natural and mathematical languages," *arXiv preprint arXiv:2405.13144*, 2024.
- [63] Z. Wang, Z. Zhu, Y. Han, Y. Lin, Z. Lin, R. Sun, and T. Ding, "Optibench: Benchmarking large language models in optimization modeling with equivalence-detection evaluation," 2024.
- [64] H. Zhai, C. Lawless, E. Vitercik, and L. Leqi, "Equivamap: Leveraging llms for automatic equivalence checking of optimization formulations," *arXiv preprint arXiv:2502.14760*, 2025.
- [65] P. P. Dakle, S. Kadioğlu, K. Uppuluri, R. Politi, P. Raghavan, S. Rallabandi, and R. Srinivasamurthy, "Ner4opt: Named entity recognition for optimization modelling from natural language," in *Proceedings of the International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, 2023, pp. 299–319.
- [66] T. Ahmed and S. Choudhury, "Lm4opt: Unveiling the potential of large language models in formulating mathematical optimization problems," *INFORS: Information Systems and Operational Research*, vol. 62, no. 4, pp. 559–572, 2024.
- [67] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [68] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [69] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier,

- L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7b,” *CoRR*, vol. abs/2310.06825, 2023.
- [70] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [71] V. Lima, D. T. Phan, J. Kalagnanam, D. Patel, and N. Zhou, “Toward a trustworthy optimization modeling agent via verifiable synthetic data generation,” *arXiv preprint arXiv:2508.03117*, 2025.
- [72] Z. Yang, Y. Wang, Y. Huang, Z. Guo, W. Shi, X. Han, L. Feng, L. Song, X. Liang, and J. Tang, “Optibench meets resocratic: Measure and improve llms for optimization modeling,” *arXiv preprint arXiv:2407.09887*, 2024.
- [73] H. Lu, Z. Xie, Y. Wu, C. Ren, Y. Chen, and Z. Wen, “Optmath: A scalable bidirectional data synthesis framework for optimization modeling,” *arXiv preprint arXiv:2502.11102*, 2025.
- [74] C. Zhou, J. Yang, L. Xin, Y. Chen, Z. He, and D. Ge, “Auto-formulating dynamic programming problems with large language models,” *arXiv preprint arXiv:2507.11737*, 2025.
- [75] T. Wang, W. Yu, Z. He, Z. Liu, H. Gong, H. Wu, X. Han, W. Shi, R. She, F. Zhu, and T. Zhong, “Bpp-search: Enhancing tree of thought reasoning for mathematical modeling problem solving,” *arXiv preprint arXiv:2411.17404*, 2024.
- [76] Y. Wu, Y. Zhang, Y. Wu, Y. Wang, J. Zhang, and J. Cheng, “Step-opt: Boosting optimization modeling in llms through iterative data synthesis and structured validation,” *arXiv preprint arXiv:2506.17637*, 2025.
- [77] C. Jiang, X. Shu, H. Qian, X. Lu, J. Zhou, A. Zhou, and Y. Yu, “Llmopt: Learning to define and solve general optimization problems from scratch,” *arXiv preprint arXiv:2410.13213*, 2024.
- [78] Y. Chen, J. Xia, S. Shao, D. Ge, and Y. Ye, “Solver-informed rl: Grounding large language models for authentic optimization modeling,” *arXiv preprint arXiv:2505.11792*, 2025.
- [79] P. T. Amarasinghe, S. Nguyen, Y. Sun, and D. Alahakoon, “Ai-copilot for business optimisation: A framework and a case study in production scheduling,” *arXiv preprint arXiv:2309.13218*, 2023.
- [80] C. Zhou, T. Xu, J. Lin, and D. Ge, “Steporlm: A self-evolving framework with generative process supervision for operations research language models,” *arXiv preprint arXiv:2509.22558*, 2025.
- [81] P. Guo, Y. Chen, Y. Tsai, and S. Lin, “Towards optimizing with large language models,” *arXiv preprint arXiv:2310.05204*, 2023.
- [82] S. Jiang, M. Xie, and J. Luo, “Large language models for combinatorial optimization of design structure matrix,” *arXiv preprint arXiv:2411.12571*, 2024.
- [83] K. Qiu, S. Bakirtzis, I. Wassell, H. Song, J. Zhang, and K. Wang, “Large language model-based wireless network design,” *IEEE Wireless Communications Letters*, vol. 13, no. 12, pp. 3340–3344, 2024.
- [84] A. Ghose, A. B. Kahng, S. Kundu, and Z. Wang, “Orfs-agent: Tool-using agents for chip design optimization,” *arXiv preprint arXiv:2506.08332*, 2025.
- [85] W. Jiang, Z. Wang, J. Zhai, S. Ma, Z. Zhao, and C. Shen, “An optimizable suffix is worth a thousand templates: Efficient black-box jailbreaking without affirmative phrases via llm as optimizer,” *arXiv preprint arXiv:2408.11313*, 2024.
- [86] R. T. Lange, Y. Tian, and Y. Tang, “Large language models as evolution strategies,” *arXiv preprint arXiv:2402.18381*, 2024.
- [87] C. Cheng, A. Nie, and A. Swaminathan, “Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 71 596–71 642, 2024.
- [88] Y. Huang, W. Zhang, L. Feng, X. Wu, and K. C. Tan, “How multimodal integration boost the performance of llm for optimization: Case study on capacitated vehicle routing problems,” in *Proceedings of the IEEE Symposium for Multidisciplinary Computational Intelligence Incubators*, 2025, pp. 1–7.
- [89] M. Elhenawy, A. Abutahoun, T. I. Alhadidi, A. Jaber, H. I. Ashqar, S. Jaradat, A. Abdelhay, S. Glaser, and A. Rakotonirainy, “Visual reasoning and multi-agent approach in multimodal large language models (mllms): Solving tsp and mtnsp combinatorial challenges,” *arXiv preprint arXiv:2407.00092*, 2024.
- [90] J. Zhao, K. H. Cheong, and W. Pedrycz, “Bridging visualization and optimization: Multimodal large language models on graph-structured combinatorial optimization,” *arXiv preprint arXiv:2501.11968*, 2025.
- [91] T. Zhang, J. Yuan, and S. Avestimehr, “Revisiting opro: The limitations of small-scale llms as optimizers,” *arXiv preprint arXiv:2405.10276*, 2024.
- [92] H. Abgaryan, A. Harutyunyan, and T. Cazenave, “Llms can schedule,” *arXiv preprint arXiv:2408.06993*, 2024.
- [93] A. Chen, S. D. Stanton, F. Ding, R. G. Alberstein, A. M. Watkins, R. Bonneau, V. Gligorijević, K. Cho, and N. C. Frey, “Generalists vs. specialists: Evaluating llms on highly-constrained biophysical sequence optimization tasks,” *arXiv preprint arXiv:2410.22296*, 2024.
- [94] Y. Xue, Y. Wang, J. Liang, and A. Slowik, “A self-adaptive mutation neural architecture search algorithm based on blocks,” *IEEE Computational Intelligence Magazine*, vol. 16, no. 3, pp. 67–78, 2021.
- [95] Y. Huang, K. Sun, Y. Ma, and R. Cheng, “Exploring neural architecture search spaces via visual analytics [application notes],” *IEEE Computational Intelligence Magazine*, vol. 20, no. 4, pp. 99–112, 2025.
- [96] C. Yu, X. Liu, Y. Wang, Y. Liu, W. Feng, X. Deng, C. Tang, and J. Lv, “Gpt-nas: Evolutionary neural architecture search with the generative pre-trained model,” *arXiv preprint arXiv:2305.05351*, 2023.
- [97] G. Jawahar, M. Abdul-Mageed, L. V. Lakshmanan, and D. Ding, “Llm performance predictors are good initializers for architecture search,” *arXiv preprint arXiv:2310.16712*, 2023.
- [98] Y. G. N. Teukam, F. Zipoli, T. Laino, E. Criscuolo, F. Grisoni, and M. Manica, “Integrating genetic algorithms and language models for enhanced enzyme design,” *arXiv preprint*, 2024.
- [99] I. De Zarzà, J. De Curtò, G. Roig, and C. T. Calafate, “Optimized financial planning: Integrating individual and cooperative budgeting models with llm recommendations,” *AI*, vol. 5, no. 1, pp. 91–114, 2023.
- [100] J. Zhao, T. Wen, and K. H. Cheong, “Can large language models be trusted as evolutionary optimizers for network-structured combinatorial problems?” *IEEE Transactions on Network Science and Engineering*, pp. 1–17, 2025.
- [101] R. Lange, Y. Tian, and Y. Tang, “Evolution transformer: In-context evolutionary optimization,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2024, pp. 575–578.
- [102] Y. Huang, X. Lv, S. Wu, J. Wu, L. Feng, and K. C. Tan, “Advancing automated knowledge transfer in evolutionary multitasking via large language models,” *arXiv preprint arXiv:2409.04270*, 2024.
- [103] E. Meyerson, M. J. Nelson, H. Bradley, A. Gaier, A. Moradi, A. K. Hoover, and J. Lehman, “Language model crossover: Variation through few-shot prompting,” *ACM Transactions on Evolutionary Learning*, vol. 4, no. 4, pp. 1–40, 2024.
- [104] S. Liu, C. Chen, X. Qu, K. Tang, and Y. S. Ong, “Large language models as evolutionary optimizers,” in *Proceedings of the IEEE Congress on Evolutionary Computation*, 2024, pp. 1–8.
- [105] S. Ali, M. Ashraf, S. Hegazy, F. Salem, H. Mokhtar, M. M. Gaber, and M. T. Alrefaie, “Pair: A novel large language model-guided selection strategy for evolutionary algorithms,” *arXiv preprint arXiv:2503.03239*, 2025.
- [106] Y. Shinohara, J. Xu, T. Li, and H. Iba, “Large language models as particle swarm optimizers,” *arXiv preprint arXiv:2504.09247*, 2025.
- [107] S. Brahmachary, S. M. Joshi, A. Panda, K. Koneripalli, A. K. Sagotra, H. Patel, A. Sharma, A. D. Jagtap, and K. Kalyanaraman, “Large language model-based evolutionary optimizer: Reasoning with elitism,” *Neurocomputing*, vol. 622, p. 129272, 2025.
- [108] F. Liu, X. Lin, S. Yao, Z. Wang, X. Tong, M. Yuan, and Q. Zhang, “Large language model for multiobjective evolutionary optimization,” in *Proceedings of the International Conference on Evolutionary Multi-Criterion Optimization*, 2025, pp. 178–191.
- [109] Q. Zhang and H. Li, “Moea/d: A multiobjective evolutionary algorithm based on decomposition,” *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [110] Z. Wang, S. Liu, J. Chen, and K. C. Tan, “Large language model-aided evolutionary search for constrained multiobjective optimization,” in *Proceedings of the International Conference on Intelligent Computing*, 2024, pp. 218–230.
- [111] W. Liu, L. Chen, and Z. Tang, “Large language model aided multi-objective evolutionary algorithm: A low-cost adaptive approach,” *arXiv preprint arXiv:2410.02301*, 2024.
- [112] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [113] M. L. Puterman, “Markov decision processes,” *Handbooks in Operations Research and Management Science*, vol. 2, pp. 331–434, 1990.
- [114] R. Chen, B. Yang, S. Li, and S. Wang, “A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem,” *Computers & Industrial Engineering*, vol. 149, p. 106778, 2020.
- [115] J. Pei, J. Liu, and Y. Mei, “Learning from offline and online experiences: A hybrid adaptive operator selection framework,” in *Proceedings*

- of the *Genetic and Evolutionary Computation Conference*, 2024, pp. 1017–1025.
- [116] Y. Zhang, G. Yi, H. Wang, Y. Cheng, Y. Chen, and Z. Wei, “An improved abc algorithm based on deep reinforcement learning for multi-uav target assignment,” in *Proceedings of the China Automation Congress*, 2024, pp. 4921–4926.
- [117] O. Kramer, “Large language models for tuning evolution strategies,” *arXiv preprint arXiv:2405.10999*, 2024.
- [118] —, “Llama tunes cma-es,” in *Proceedings of the European Symposium on Artificial Neural Networks*, 2024.
- [119] L. L. Custode, F. Caraffini, A. Yaman, and G. Iacca, “An investigation on the use of large language models for hyperparameter tuning in evolutionary algorithms,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2024, pp. 1838–1845.
- [120] Y. Zhang and G. Yi, “Laos: Large language model-driven adaptive operator selection for evolutionary algorithms,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2025, pp. 517–526.
- [121] N. van Stein and T. Bäck, “Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics,” *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 2, pp. 331–345, 2024.
- [122] H. Hao, X. Zhang, and A. Zhou, “Large language models as surrogate models in evolutionary algorithms: A preliminary study,” *Swarm and Evolutionary Computation*, vol. 91, p. 101741, 2024.
- [123] T. Nguyen and A. Grover, “Lico: Large language models for in-context molecular optimization,” *arXiv preprint arXiv:2406.18851*, 2024.
- [124] T. Rios, F. Lanfermann, and S. Menzel, “Large language model-assisted surrogate modelling for engineering optimization,” in *Proceedings of the IEEE Conference on Artificial Intelligence*, 2024, pp. 796–803.
- [125] L. Xie, G. Li, Z. Wang, E. Chung, and M. Gong, “Large language model-driven surrogate-assisted evolutionary algorithm for expensive optimization,” *arXiv preprint arXiv:2507.02892*, 2025.
- [126] X. Zhang, Y. Gong, and J. Zhang, “Large language model as meta-surrogate for data-driven many-task optimization: A proof-of-principle study,” *arXiv preprint arXiv:2503.08301*, 2025.
- [127] X. Wu, J. Wu, Y. Zhou, L. Feng, and K. C. Tan, “Towards robustness and explainability of automatic algorithm selection,” in *Proceedings of the International Conference on Machine Learning*, 2025.
- [128] S. M. Abdulrahman, P. Brazdil, J. N. Van Rijn, and J. Vanschoren, “Speeding up algorithm selection using average ranking and active testing by introducing runtime,” *Machine Learning*, vol. 107, no. 1, pp. 79–108, 2018.
- [129] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown, “Satzilla: Portfolio-based algorithm selection for sat,” *Journal of Artificial Intelligence Research*, vol. 32, pp. 565–606, 2008.
- [130] L. Fehring, J. Hanselle, and A. Tornede, “Harris: Hybrid ranking and regression forests for algorithm selection,” *arXiv preprint arXiv:2210.17341*, 2022.
- [131] X. Wu, Y. Zhong, J. Wu, B. Jiang, and K. C. Tan, “Large language model-enhanced algorithm selection: Towards comprehensive algorithm representation,” *arXiv preprint arXiv:2311.13184*, 2023.
- [132] S. Zhang, S. Liu, N. Lu, J. Wu, J. Liu, Y. S. Ong, and K. Tang, “Llm-driven instance-specific heuristic generation and selection,” *arXiv preprint arXiv:2506.00490*, 2025.
- [133] W. Yi, R. Qu, L. Jiao, and B. Niu, “Automated design of metaheuristics using reinforcement learning within a novel general search framework,” *IEEE Transactions on Evolutionary Computation*, vol. 27, no. 4, pp. 1072–1084, 2022.
- [134] M. Pluhacek, A. Kazikova, T. Kadavy, A. Viktorin, and R. Senkerik, “Leveraging large language models for the generation of novel meta-heuristic optimization algorithms,” in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, 2023, pp. 1812–1820.
- [135] R. Zhong, Y. Xu, C. Zhang, and J. Yu, “Leveraging large language model to generate a novel metaheuristic algorithm with crisper framework,” *Cluster Computing*, vol. 27, no. 10, pp. 13 835–13 869, 2024.
- [136] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.
- [137] F. Liu, X. Tong, M. Yuan, and Q. Zhang, “Algorithm evolution using large language model,” *arXiv preprint arXiv:2311.15249*, 2023.
- [138] F. Liu, R. Zhang, Z. Xie, R. Sun, K. Li, X. Lin, Z. Wang, Z. Lu, and Q. Zhang, “Llm4ad: A platform for algorithm design with large language model,” *arXiv preprint arXiv:2412.17287*, 2024.
- [139] F. Liu, Q. Zhang, X. Tong, K. Mao, and M. Yuan, “Fitness landscape of large language model-assisted automated algorithm search,” *arXiv preprint arXiv:2504.19636*, 2025.
- [140] F. Liu, R. Zhang, X. Lin, Z. Lu, and Q. Zhang, “Fine-tuning large language model for automated algorithm design,” *arXiv preprint arXiv:2507.10614*, 2025.
- [141] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang, “Multi-objective evolution of heuristic using large language model,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, pp. 27 144–27 152.
- [142] F. Liu, Y. Liu, Q. Zhang, X. Tong, and M. Yuan, “Eoh-s: Evolution of heuristic set using llms for automated heuristic design,” *arXiv preprint arXiv:2508.03082*, 2025.
- [143] W. Yatong, P. Yuchen, and Z. Yuqi, “Ts-eoh: An edge server task scheduling algorithm based on evolution of heuristic,” *arXiv preprint arXiv:2409.09063*, 2024.
- [144] Y. Yao, F. Liu, J. Cheng, and Q. Zhang, “Evolve cost-aware acquisition functions using large language models,” in *Proceedings of the International Conference on Parallel Problem Solving from Nature*, 2024, pp. 374–390.
- [145] P. Guo, F. Liu, X. Lin, Q. Zhao, and Q. Zhang, “L-autoda: Large language models for automatically evolving decision-based adversarial attacks,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2024, pp. 1846–1854.
- [146] N. van Stein and T. Bäck, “Llamea: Automatically generating meta-heuristics with large language models,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2025, pp. 81–82.
- [147] N. van Stein, A. V. Kononova, L. Kotthoff, and T. Bäck, “Code evolution graphs: Understanding large language model driven design of algorithms,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2025, pp. 943–951.
- [148] N. van Stein, H. Yin, A. V. Kononova, T. Bäck, and G. Ochoa, “Behaviour space analysis of llm-driven meta-heuristic discovery,” *arXiv preprint arXiv:2507.03605*, 2025.
- [149] N. van Stein, A. V. Kononova, H. Yin, and T. Bäck, “Blade: Benchmark suite for llm-driven automated design and evolution of iterative optimisation heuristics,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2025, pp. 2336–2344.
- [150] N. van Stein, D. Vermetten, and T. Bäck, “In-the-loop hyper-parameter optimization for llm-based automated design of heuristics,” *ACM Transactions on Evolutionary Learning*, 2024.
- [151] W. Li, N. van Stein, T. Bäck, and E. Raponi, “Llamea-bo: A large language model evolutionary algorithm for automatically generating bayesian optimization algorithms,” *arXiv preprint arXiv:2505.21034*, 2025.
- [152] H. Yin, A. V. Kononova, T. Bäck, and N. van Stein, “Optimizing photonic structures with large language model driven algorithm discovery,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2025, pp. 2354–2362.
- [153] Y. Sun, F. Ye, X. Zhang, S. Huang, B. Zhang, K. Wei, and S. Cai, “Autosat: Automatically optimize sat solvers via large language models,” *arXiv preprint arXiv:2402.10705*, 2024.
- [154] Y. Huang, S. Wu, W. Zhang, J. Wu, L. Feng, and K. C. Tan, “Autonomous multi-objective optimization using large language model,” *IEEE Transactions on Evolutionary Computation*, 2025.
- [155] R. Zhang, F. Liu, X. Lin, Z. Wang, Z. Lu, and Q. Zhang, “Understanding the importance of evolutionary search in automated heuristic design with large language models,” in *Proceedings of the International Conference on Parallel Problem Solving from Nature*, 2024, pp. 185–202.
- [156] H. Yin, A. V. Kononova, T. Bäck, and N. van Stein, “Controlling the mutation in large language models for the efficient evolution of algorithms,” in *Proceedings of the International Conference on the Applications of Evolutionary Computation*, 2025, pp. 403–417.
- [157] R. Li, L. Wang, H. Sang, L. Yao, and L. Pan, “Llm-assisted automatic memetic algorithm for lot-streaming hybrid job shop scheduling with variable sublots,” *IEEE Transactions on Evolutionary Computation*, 2025.
- [158] H. Ling, S. Parashar, S. Khurana, B. Olson, A. Basu, G. Sinha, Z. Tu, J. Caverlee, and S. Ji, “Complex llm planning via automated heuristics discovery,” *arXiv preprint arXiv:2502.19295*, 2025.
- [159] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, “Reevo: Large language models as hyper-heuristics with reflective evolution,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 43 571–43 608, 2024.

- [160] Z. Zheng, Z. Xie, Z. Wang, and B. Hooi, “Monte carlo tree search for comprehensive exploration in llm-based automatic heuristic design,” *arXiv preprint arXiv:2501.08603*, 2025.
- [161] P. V. T. Dat, L. Doan, and H. T. T. Binh, “Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, pp. 26931–26938.
- [162] K. Michailidis, D. Tsouros, and T. Guns, “Cp-bench: Evaluating large language models for constraint modelling,” *arXiv preprint arXiv:2506.06052*, 2025.
- [163] H. Liu, J. Wang, Y. Cai, X. Han, Y. Kuang, and J. Hao, “Optitree: Hierarchical thoughts generation with tree search for llm optimization modeling,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 120713–120781, 2026.
- [164] Z. Xiao, Y. J. Wang, X. Han, S. Guan, J. Zhu, J. Xie, L. Xu, H. Wu, W. Y. Yu, Z. Liu *et al.*, “Deepor: A deep reasoning foundation model for optimization modeling,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026, pp. 34052–34060.
- [165] L. Xing, X. Wang, Y. Feng, Z. Fan, J. Xiong, Z. Guo, X. Fu, R. Ramamonjison, M. Mostajabdeh, X. Han *et al.*, “Towards human-aligned evaluation for linear programming word problems,” in *Proceedings of the Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, 2024, pp. 16550–16556.
- [166] H. Wang, S. Feng, T. He, Z. Tan, X. Han, and Y. Tsvetkov, “Can language models solve graph problems in natural language?” *Advances in Neural Information Processing Systems*, vol. 36, pp. 30840–30861, 2023.
- [167] M. Aghzal, E. Plaku, and Z. Yao, “Can large language models be good path planners? a benchmark and investigation on spatial-temporal reasoning,” *arXiv preprint arXiv:2310.03249*, 2023.
- [168] J. Tang, Q. Zhang, Y. Li, N. Chen, and J. Li, “Grapharena: Evaluating and exploring large language models on graph computation,” in *Proceedings of the International Conference on Learning Representations*, 2025, pp. 48118–48145.
- [169] X. Li, J. Chen, X. Fang, S. Ding, H. Duan, Q. Liu, and K. Chen, “Opt-bench: Evaluating llm agent on large-scale search spaces optimization problems,” *arXiv preprint arXiv:2506.10764*, 2025.
- [170] W. Sun, S. Feng, S. Li, and Y. Yang, “Co-bench: Benchmarking language model agents in algorithm search for combinatorial optimization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026, pp. 33126–33134.
- [171] S. Feng, W. Sun, S. Li, A. Talwalkar, and Y. Yang, “Frontierco: Real-world and large-scale evaluation of machine learning solvers for combinatorial optimization,” in *Proceedings of the International Conference on Learning Representations*, 2026.
- [172] H. Chen, Y. Wang, Y. Cai, H. Hu, J. Li, S. Huang, C. Deng, R. Liang, S. Kong, H. Ren *et al.*, “Heurigym: An agentic benchmark for llm-crafted heuristics in combinatorial optimization,” *arXiv preprint arXiv:2506.07972*, 2025.
- [173] Y. Imajuku, K. Horie, Y. Iwata, K. Aoki, N. Takahashi, and T. Akiba, “Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering,” *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [174] J. Wei, X. Wang, D. Schuurmans, M. Bosma, I. Brian, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [175] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, and D. Brockhoff, “Coco: A platform for comparing continuous optimizers in a black-box setting,” *Optimization Methods and Software*, vol. 36, no. 1, pp. 114–144, 2021.
- [176] S. Das and P. N. Suganthan, “Differential evolution: A survey of the state-of-the-art,” *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2010.
- [177] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of the International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.
- [178] N. Hansen, S. D. Müller, and P. Koumoutsakos, “Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es),” *Evolutionary Computation*, vol. 11, no. 1, pp. 1–18, 2003.
- [179] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, “Deep reinforcement learning based parameter control in differential evolution,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, pp. 709–717.
- [180] R. Lange, T. Schaul, Y. Chen, T. Zahavy, V. Dalibard, C. Lu, S. Singh, and S. Flennerhag, “Discovering evolution strategies via meta-black-box optimization,” in *Proceedings of the companion conference on genetic and evolutionary computation*, 2023, pp. 29–30.
- [181] A. Chen, D. Dohan, and D. So, “Evoprompting: Language models for code-level neural architecture search,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 7787–7817, 2023.
- [182] P. Wang, Z. Zhao, H. Wen, F. Wang, B. Wang, Q. Zhang, and Y. Wang, “Llm-autoda: Large language model-driven automatic data augmentation for long-tailed problems,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 64915–64941, 2024.
- [183] X. Li, X. Sun, A. Wang, J. Li, and C. Shum, “Cuda-1l: Improving cuda optimization via contrastive reinforcement learning,” *arXiv preprint arXiv:2507.14111*, 2025.
- [184] Z. Ma, H. Guo, J. Chen, G. Peng, Z. Cao, Y. Ma, and Y. Gong, “Llamoco: Instruction tuning of large language models for optimization code generation,” *arXiv preprint arXiv:2403.01131*, 2024.
- [185] F. Wang, H. Liu, Z. Dai, J. Zeng, Z. Zhang, Z. Wu, C. Luo, Z. Li, X. Tang, Q. He, and S. Wang, “Agenttts: Large language model agent for test-time compute-optimal scaling strategy in complex tasks,” *arXiv preprint arXiv:2508.00890*, 2025.
- [186] R. Zhong, Y. Cao, J. Yu, and M. Munetomo, “Large language model assisted adversarial robustness neural architecture search,” in *Proceedings of the International Conference on Data-driven Optimization of Complex Systems*, 2024, pp. 433–437.
- [187] Y. Yu and J. Zutti, “Llm-guided evolution: An autonomous model optimization for object detection,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2025, pp. 2363–2370.
- [188] M. U. Nasir, S. Earle, J. Togelius, S. James, and C. Cleghorn, “Llmatic: Neural architecture search via large language models and quality diversity optimization,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 1110–1118.
- [189] C. Morris, M. Jurado, and J. Zutti, “Llm guided evolution-the automation of models advancing models,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 377–384.
- [190] S. Mo, K. Wu, Q. Gao, X. Teng, and J. Liu, “Autosgmn: Automatic propagation mechanism discovery for spectral graph neural networks,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, pp. 19493–19502.
- [191] X. Li, C. Zhang, J. Wang, F. Wu, Y. Li, and X. Jin, “Efficient and stealthy jailbreak attacks via adversarial prompt distillation from llms to slms,” *arXiv preprint arXiv:2506.17231*, 2025.
- [192] M. Yu, J. Fang, Y. Zhou, X. Fan, K. Wang, S. Pan, and Q. Wen, “Llm-virus: Evolutionary jailbreak attack on large language models,” *arXiv preprint arXiv:2501.00055*, 2024.
- [193] Z. Wang, K. Zhang, Z. Zhao, Y. Wen, A. Pandey, H. Liu, and K. Ding, “A survey of large language models for text-guided molecular discovery: From molecule generation to optimization,” *arXiv preprint arXiv:2505.16094*, 2025.
- [194] Y. G. Nana Teukam, F. Zipoli, T. Laino, E. Criscuolo, F. Grisoni, and M. Manica, “Integrating genetic algorithms and language models for enhanced enzyme design,” *Briefings in Bioinformatics*, vol. 26, no. 1, p. bbae675, 2025.
- [195] Y. Zhang, K. Zheng, F. Liu, Q. Zhang, and Z. Wang, “Autoturb: Using large language models for automatic algebraic turbulence model discovery,” *Physics of Fluids*, vol. 37, no. 1, 2025.
- [196] H. Wang, M. Skreta, C.-T. Ser, W. Gao, L. Kong, F. Strieth-Kalthoff, C. Duan, Y. Zhuang, Y. Yu, Y. Zhu, Y. Du, A. Aspuru-Guzik, K. Neklyudov, and C. Zhang, “Efficient evolutionary search over chemical space with large language models,” *arXiv preprint arXiv:2406.16976*, 2024.
- [197] P. Guevorguian, M. Bedrosian, T. Fahradyan, G. Chilingaryan, H. Khachatryan, and A. Aghajanyan, “Small molecule optimization with large language models,” *arXiv preprint arXiv:2407.18897*, 2024.
- [198] N. Ran, Y. Wang, and R. Allmendinger, “Mollm: Multi-objective large language model for molecular design—optimizing with experts,” *arXiv preprint arXiv:2502.12845*, 2025.
- [199] T. V. Tran and T. S. Hy, “Protein design by directed evolution guided by large language models,” *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 2, pp. 418–428, 2024.
- [200] Y. Wang, J. He, Y. Du, X. Chen, J. C. Li, L. Liu, X. Xu, and S. Hassoun, “Large language model is secretly a protein sequence optimizer,” *arXiv preprint arXiv:2501.09274*, 2025.
- [201] A. Chen, S. D. Stanton, R. G. Alberstein, A. M. Watkins, R. Bonneau, V. Gligorijevic, K. Cho, and N. C. Frey, “Llms are highly-constrained biophysical sequence optimizers,” in *Proceedings of the Workshop on AI for New Drug Modalities at the Conference on Neural Information Processing Systems*, 2024.

- [202] L. Lv, Z. Lin, H. Li, Y. Liu, J. Cui, C. Y.-C. Chen, L. Yuan, and Y. Tian, "Prollama: A protein large language model for multi-task protein language processing," *IEEE Transactions on Artificial Intelligence*, 2025.
- [203] X. Zhang, Z. Xu, G. Zhu, C. M. J. Tay, Y. Cui, B. C. Khoo, and L. Zhu, "Using large language models for parametric shape optimization," *Physics of Fluids*, vol. 37, no. 8, 2025.
- [204] R. Li, C. Zhang, S. Mao, H. Huang, M. Zhong, Y. Cui, X. Zhou, F. Yin, S. Theodoridis, and Z. Zhang, "From english to pcell: Llm helps design and optimize photonic crystal surface emitting lasers," 2023.
- [205] M. Du, Y. Chen, Z. Wang, L. Nie, and D. Zhang, "Large language models for automatic equation discovery of nonlinear dynamics," *Physics of Fluids*, vol. 36, no. 9, 2024.
- [206] R. Sun, C. Chen, F. Guo, and K. Liu, "Integrating llms and evolutionary algorithms for spin glass optimization," in *Proceedings of the International Conference on Artificial Intelligence and Industrial Technology Applications*, 2025, pp. 1271–1274.
- [207] P. Ma, T.-H. Wang, M. Guo, Z. Sun, J. B. Tenenbaum, D. Rus, C. Gan, and W. Matusik, "Llm and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery," *arXiv preprint arXiv:2405.09783*, 2024.
- [208] X. Peng, Y. Liu, Y. Cang, C. Cao, and M. Chen, "Llm-optimira: Llm-driven optimization of resource allocation for non-convex problems in wireless communications," *arXiv preprint arXiv:2505.02091*, 2025.
- [209] J. Wen, C. Su, J. Kang, J. Nie, Y. Zhang, J. Tang, D. Niyato, and C. Yuen, "Hybridrag-based llm agents for low-carbon optimization in low-altitude economy networks," *arXiv preprint arXiv:2506.15947*, 2025.
- [210] W. Lee and J. Park, "Llm-empowered resource allocation in wireless communications systems," *arXiv preprint arXiv:2408.02944*, 2024.
- [211] H. Zhou, C. Hu, D. Yuan, Y. Yuan, D. Wu, X. Liu, and C. Zhang, "Large language model (llm)-enabled in-context learning for wireless network optimization: A case study of power control," *arXiv preprint arXiv:2408.00214*, 2024.
- [212] Y. Wang, J. Farooq, H. Ghazzai, and G. Setti, "Multi-uav placement for integrated access and backhauling using llm-driven optimization," in *Proceedings of the IEEE Wireless Communications and Networking Conference*, 2025, pp. 1–6.
- [213] H. Li, M. Xiao, K. Wang, D. I. Kim, and M. Debbah, "Large language model based multi-objective optimization for integrated sensing and communications in uav networks," *IEEE Wireless Communications Letters*, vol. 14, no. 4, pp. 979–983, 2025.
- [214] J. Hou, K. Qiu, Z. Zhang, Y. Yu, K. Wang, S. Capolongo, J. Zhang, Z. Li, and J. Zhang, "Wireless-friendly window position optimization for ris-aided outdoor-to-indoor networks based on multi-modal large language model," *arXiv preprint arXiv:2410.20691*, 2024.
- [215] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 7, no. 4, pp. 1051–1064, 2022.
- [216] H. Guo, Y. Ma, Z. Ma, J. Chen, X. Zhang, Z. Cao, J. Zhang, and Y. Gong, "Deep reinforcement learning for dynamic algorithm selection: A proof-of-principle study on differential evolution," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 7, pp. 4247–4259, 2024.
- [217] Y. Liu, F. Wu, Z. Liu, K. Wang, F. Wang, and X. Qu, "Can language models be used for real-world urban-delivery route optimization?" *The Innovation*, vol. 4, no. 6, 2023.
- [218] D. Jobson and Y. Li, "Investigating the potential of using large language models for scheduling," in *Proceedings of the ACM International Conference on AI-Powered Software*, 2024, pp. 170–171.
- [219] H. Chen, G. E. Constante-Flores, and C. Li, "Diagnosing infeasible optimization problems using large language models," *INFOR: Information Systems and Operational Research*, vol. 62, no. 4, pp. 573–587, 2024.
- [220] G. Singh and K. K. Bali, "Enhancing decision-making in optimization through llm-assisted inference: A neural networks perspective," in *Proceedings of the International Joint Conference on Neural Networks*, 2024, pp. 1–7.
- [221] C. Chacón Sartori, C. Blum, and G. Ochoa, "Large language models for the automated analysis of optimization algorithms," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 160–168.
- [222] P. Maddigan, A. Lensen, and B. Xue, "Explaining genetic programming trees using large language models," *arXiv preprint arXiv:2403.03397*, 2024.
- [223] D. Kikuta, H. Ikeuchi, K. Tajiri, and Y. Nakano, "Routeexplainer: An explanation framework for vehicle routing problem," in *Proceedings of the Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2024, pp. 30–42.
- [224] C. Li, R. Yang, T. Li, M. Bafarassat, K. Sharifi, D. Bergemann, and Z. Yang, "Stride: A tool-assisted llm agent framework for strategic and interactive decision-making," *arXiv preprint arXiv:2405.16376*, 2024.
- [225] B. Huang, R. Cheng, and K. C. Tan, "Evogit: Decentralized code evolution via git-based multi-agent collaboration," *arXiv preprint arXiv:2506.02049*, 2025.
- [226] S. Liu, S. Agarwal, M. Maheswaran, M. Cemri, Z. Li, Q. Mang, A. Naren, E. Boneh, A. Cheng, M. Z. Pan *et al.*, "Evovox: Meta-evolution for automated discovery," *arXiv preprint arXiv:2602.23413*, 2026.
- [227] S. Alzubi, N. Provenzano, J. Bingham, W. Chen, and T. Vu, "Evoskill: Automated skill discovery for multi-agent systems," *arXiv preprint arXiv:2603.02766*, 2026.
- [228] R. Cheng and Y. Jin, "A competitive swarm optimizer for large scale optimization," *IEEE Transactions on Cybernetics*, vol. 45, no. 2, pp. 191–204, 2014.
- [229] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "Platemo: A matlab platform for evolutionary multi-objective optimization [educational forum]," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.
- [230] J. H. Holland, "Genetic algorithms," *Scientific American*, vol. 267, no. 1, pp. 66–73, 1992.
- [231] J. R. Koza, "Genetic programming as a means for programming computers by natural selection," *Statistics and Computing*, vol. 4, no. 2, pp. 87–112, 1994.
- [232] A. Arias-Montano, C. A. C. Coello, and E. Mezura-Montes, "Multiobjective evolutionary algorithms in aeronautical and aerospace engineering," *IEEE Transactions on Evolutionary Computation*, vol. 16, no. 5, pp. 662–694, 2012.
- [233] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.
- [234] E.-G. Talbi, "Machine learning into metaheuristics: A survey and taxonomy," *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–32, 2021.
- [235] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [236] P. Cai, Y. Fan, and F. Leu, "Compare encoder-decoder, encoder-only, and decoder-only architectures for text generation on low-resource datasets," in *Proceedings of the International Conference on Broadband and Wireless Computing, Communication and Applications*, 2021, pp. 216–225.
- [237] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.
- [238] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [239] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "Bert: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.
- [240] L. Floridi and M. Chiriatti, "Gpt-3: its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, no. 4, pp. 681–694, 2020.
- [241] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "Deepseek-coder: when the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [242] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. El-nashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.
- [243] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *arXiv preprint arXiv:2403.14608*, 2024.
- [244] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 199–22 213, 2022.
- [245] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with

- large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 11 809–11 822, 2023.
- [246] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, “Graph of thoughts: solving elaborate problems with large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, pp. 17 682–17 690.
 - [247] C. Li, J. Liang, A. Zeng, X. Chen, K. Hausman, D. Sadigh, S. Levine, L. Fei-Fei, F. Xia, and B. Ichter, “Chain of code: Reasoning with a language model-augmented code emulator,” *arXiv preprint arXiv:2312.04474*, 2023.
 - [248] X. Li, R. Zhao, Y. K. Chia, B. Ding, S. Joty, S. Poria, and L. Bing, “Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources,” *arXiv preprint arXiv:2305.13269*, 2023.
 - [249] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 892–34 916, 2023.
 - [250] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
 - [251] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
 - [252] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 53 728–53 741, 2023.
 - [253] Z. Ding, Z. Tan, J. Zhang, and T. Chen, “Or-r1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026, pp. 228–236.
 - [254] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, “Evbox: A distributed gpu-accelerated framework for scalable evolutionary computation,” *IEEE Transactions on Evolutionary Computation*, 2024.
 - [255] G. Cenikj, A. Nikolikj, G. Petelin, N. Van Stein, C. Doerr, and T. Eftimov, “A survey of features used for representing black-box single-objective continuous optimization,” *Swarm and Evolutionary Computation*, vol. 101, p. 102288, 2026.
 - [256] C. Feng, M. Chen, Z. Li, and R. Cheng, “Unleashing the potential of differential evolution through individual-level strategy diversity,” *arXiv preprint arXiv:2602.01147*, 2026.

A Systematic Survey on Large Language Models for Evolutionary Optimization: From Modeling to Solving (Supplementary Document)

A BACKGROUND

Research on LLM-enabled component control, high-level orchestration, and algorithm generation has focused primarily on evolutionary algorithms (EAs). Accordingly, this section provides the background required for the survey. Section A-A introduces the common framework and major paradigms of EAs, while Section A-B summarizes LLM architectures and the principal techniques used to adapt them.

A. Evolutionary Algorithms

Evolutionary algorithms (EAs) are population-based, gradient-free optimization methods inspired by biological evolution. A typical EA begins by initializing a population of candidate solutions, whose representations may take the form of numerical vectors, permutations, program trees, or neural networks. At each iteration, a fitness function evaluates the candidates. Parent-selection and variation operators, such as crossover and mutation, then generate offspring, after which environmental selection constructs the next population from the parents and offspring. Many EAs also retain elite candidates to preserve the best solutions found so far. This cycle continues until a termination criterion is met, and the best-performing candidate is returned as the final solution.

Over several decades, this general framework has produced a broad range of established paradigms [228], [229], including Genetic Algorithms (GAs) [230], Genetic Programming (GP) [231], Differential Evolution (DE) [176], Particle Swarm Optimization (PSO) [177], and Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [178]. Although these paradigms share a population-based iterative structure, they employ distinct search mechanisms for different problem domains. GAs emulate natural selection through genetic operators and are well suited to discrete and combinatorial problems. DE and PSO are widely used for continuous optimization: DE generates offspring by adding weighted differences between population vectors, whereas PSO updates each particle according to its personal best and the population’s global best. CMA-ES adapts the full covariance matrix of its mutation distribution and performs particularly well on ill-conditioned continuous landscapes. GP uses tree-structured representations to support automatic programming and symbolic regression.

Compared with exact optimization methods, EAs are often better suited to non-convex, multimodal, and noisy problems. Their applications range from aerospace design [232] to hyperparameter optimization in deep learning [233]. However, the No Free Lunch (NFL) theorem [7] establishes that no optimizer is universally superior across all problem classes. Practitioners must therefore invest substantial effort in selecting, configuring, and designing appropriate EA variants. This challenge has motivated the integration of EAs with machine learning (ML) [234], including reinforcement-learning-based frameworks such as MOEA/D-DQN [215] and SYMBOL [19]. More recently, researchers have investigated how LLMs can support EAs and optimization more broadly [31], from problem formulation [27] to automated algorithm design [35].

B. Large Language Models

Large language models (LLMs) are built primarily on the Transformer architecture [235], which models global input-output dependencies through self-attention and supports parallel training on large corpora. Three principal architectural paradigms have emerged: encoder-decoder, encoder-only, and decoder-only models [236]. Encoder-decoder models, exemplified by BART [237] and T5 [238], follow the original Transformer design and are effective for sequence-to-sequence tasks. Encoder-only models, such as BERT [239], emphasize contextual representation learning for language understanding. Decoder-only models, including ChatGPT [240] and DeepSeek [241], generate text autoregressively and dominate current research on LLMs for optimization because of their strong generation and reasoning capabilities. These models are commonly adapted through two routes: prompt engineering [242] and fine-tuning [243].

Prompt engineering steers LLM outputs through carefully designed instructions without modifying model parameters. Zero-shot and few-shot prompting [244] use pretrained knowledge either directly or through a small number of in-context examples. Structured reasoning methods extend this approach. Chain-of-Thought (CoT) prompting [174] elicits intermediate reasoning steps for complex tasks, while Tree-of-Thought (ToT) [245] and Graph-of-Thought (GoT) [246] organize candidate reasoning paths as searchable trees or graphs. Retrieval-augmented generation (RAG) supplements the model’s parametric knowledge with external information at inference time. Domain-specific prompting strategies, including Chain-of-Code (CoC) [247] and Chain-of-Knowledge (CoK) [248], further tailor the reasoning process to particular task structures.

Fine-tuning updates model parameters on task-specific data. This process enables deeper adaptation but requires additional labeled data and computational resources. Parameter-efficient fine-tuning (PEFT) methods, such as Low-Rank Adaptation (LoRA), reduce this cost by updating only a small subset of parameters while keeping the backbone fixed. Two widely

used fine-tuning strategies are instruction tuning and preference alignment. Instruction fine-tuning [249] reformulates diverse tasks as instruction-input-output triplets and trains the model to follow those instructions. This approach improves zero-shot generalization, as demonstrated by Flan-T5 [250]. Alignment fine-tuning seeks to make model behavior consistent with human intentions and preferences. Reinforcement Learning with Human Feedback (RLHF) [251] optimizes a reward model derived from human judgments, whereas Direct Preference Optimization (DPO) [252] directly optimizes the likelihood of preferred outputs. Together, these techniques support the development of more reliable and useful LLMs.

B REPRESENTATIVE WORKS

This section consolidates the detailed taxonomies of the studies reviewed in the main text. Table II summarizes representative work on LLMs for optimization modeling, including each method’s objective and methodological category. Table III catalogs studies on LLMs for optimization solving and classifies them according to the role of the LLM in the optimization workflow, such as direct optimizer, low-level assistant, or high-level algorithm generator.

TABLE II
REPRESENTATIVE STUDIES ON LLMs FOR OPTIMIZATION MODELING, GROUPED INTO PROMPT-BASED AND LEARNING-BASED METHODS.

	Method	Venue	Type	Technical Summary
Prompt-based Methods	Ner4OPT [65]	CPAIOR, 2023	Two-stage	Fine-tune models for named-entity recognition with established NLP techniques.
	AOMG [45]	GECCO, 2023	Direct	Use LLMs to generate mathematical optimization models directly.
	HG 2.0 [46]	arXiv, 2023	Two-stage	Embed LLMs in a two-stage optimization-modeling framework.
	AMGPT [38]	arXiv, 2023	Two-stage	Use fine-tuned models to classify constraints before model generation.
	CoE [50]	ICLR, 2023	Multi-agent	Construct dynamic reasoning chains with 11 specialized agents.
	OptiMUS [51]	ICML, 2023	Multi-agent	Coordinate modeling, programming, and evaluation through a conductor agent.
	MAMS [54]	INFOR, 2024	Multi-agent	Use inter-agent cross-validation instead of solver-dependent verification.
	EC [47]	SGC, 2024	Two-stage	Apply a two-stage modeling framework to energy-management systems.
	CAFA [59]	NeurIPS, 2024	Two-stage	Improve model generation through code-based formalization.
	MAMO [62]	NAACL, 2024	Two-stage	Extend optimization-modeling evaluation to ordinary differential equations.
	NL2OR [61]	arXiv, 2024	Two-stage	Constrain LLM outputs with predefined abstract model structures.
	TRIP-PAL [48]	arXiv, 2024	Two-stage	Apply a two-stage modeling framework to travel planning.
	OptiLLM [56]	arXiv, 2024	Interactive	Support both single-turn and interactive input modes.
	OptiMUS-0.3 [52]	arXiv, 2024	Multi-agent	Add self-correction and structure-aware modeling to OptiMUS.
	MeetMate [57]	TiS, 2024	Interactive	Process user input interactively through five selectable task modes.
	OptiBench [63]	arXiv, 2024	Two-stage	Verify model equivalence with a modified Weisfeiler-Lehman graph-isomorphism procedure.
	VL [58]	CUI, 2025	Interactive	Translate user priorities into optimization constraints through dialogue.
	LLM-MCTS [60]	arXiv, 2025	Two-stage	Search the formulation hypothesis space with hierarchical Monte Carlo tree search.
	EquivaMap [64]	arXiv, 2025	Two-stage	Generate variable-mapping functions with LLMs and lightweight verification.
	MAP [55]	arXiv, 2025	Multi-agent	Use multiple independent reviewer agents to assess generated models.
Learning-based Methods	ORMind [53]	arXiv, 2025	Multi-agent	Replace conductor-based coordination with structured and predictable workflows.
	LEAN-LLM-OPT [49]	SSRN, 2025	Multi-agent	Use RAG for problem classification and construct instances through query operations.
	OptiTree [163]	NeurIPS, 2026	Multi-agent	Decompose problems hierarchically in a tree and synthesize the resulting subproblem analyses.
	LM4OPT [66]	INFOR, 2024	Fine-tuning	Progressively fine-tune models on the NL4OPT dataset.
	ORLM [27]	OR, 2024	Data synthesis	Expand and augment training data before fine-tuning open-source models.
	ReSocratic [72]	ICLR, 2024	Data synthesis	Introduce inverse data synthesis and construct the OPTIBENCH benchmark.
	LLMOPT [77]	ICLR, 2024	Fine-tuning	Combine model alignment with self-correction to reduce hallucinated formulations.
	BPP-Search [75]	arXiv, 2024	Data synthesis	Recover missing intermediate details during synthetic-data generation.
	OptMATH [73]	arXiv, 2025	Data synthesis	Develop a scalable bidirectional data-synthesis pipeline.
	LLMBO [79]	arXiv, 2025	Data synthesis	Fine-tune cost-efficient LLMs for domain-specific business optimization tasks.
	SIRL [78]	arXiv, 2025	Fine-tuning	Use external optimization solvers as verifiable reward evaluators for reinforcement learning.
	Step-Opt [76]	arXiv, 2025	Data synthesis	Increase problem complexity through iterative problem generation.
	DPLM [74]	arXiv, 2025	Data synthesis	Combine the diversity of forward generation with the reliability of inverse generation.
	OptiTrust [71]	arXiv, 2025	Data synthesis	Construct a verifiable pipeline for synthetic-data generation.
	StepORLM [80]	arXiv, 2025	Fine-tuning	Refine reasoning trajectories through generative process supervision and co-evolution.
	OR-R1 [253]	AAAI, 2026	Fine-tuning	Combine fine-tuning with TGRPO to improve efficiency and output consistency on unlabeled data.
	DeepOR [164]	AAAI, 2026	Fine-tuning	Develop a reasoning foundation model that generates structured optimization formulations.

TABLE III
REPRESENTATIVE STUDIES ON LLMs FOR OPTIMIZATION SOLVING, GROUPED INTO LLMs AS DIRECT OPTIMIZERS, LOW-LEVEL ASSISTANTS, AND HIGH-LEVEL ALGORITHM GENERATORS.

	Method	Venue	Type	Technical Summary
LLMs as Optimizers	OPRO [28]	ICLR, 2023	Prompt-based	Refine candidate solutions iteratively from problem descriptions and optimization trajectories.
	toLLM [81]	KDD, 2023	Prompt-based	Define four canonical tasks to evaluate the optimization limits of LLMs.
	EvoLLM [86]	GECCO, 2024	Prompt-based	Replace raw optimization trajectories with rankings of candidate quality.
	MLLMO [88]	MCII, 2024	Prompt-based	Use multimodal LLMs to process problem descriptions and route visualizations jointly for CVRPs.
	POM [21]	NeurIPS, 2024	Learning-based	Pretrain a general-purpose foundation model for zero-shot black-box optimization.
	OPTO [87]	NeurIPS, 2024	Prompt-based	Replace conventional optimization trajectories with rich execution traces.
	VRMA [89]	arXiv, 2024	Prompt-based	Use multimodal LLMs to process two-dimensional point-distribution maps.
	ROPRO [91]	ACL, 2024	Prompt-based	Analyze OPRO’s model dependence and its limitations on smaller models.
	BBOLL [30]	arXiv, 2024	Prompt-based	Evaluate LLMs on discrete and continuous black-box optimization tasks.
	LLMS [92]	arXiv, 2024	Learning-based	Fine-tune LLMs on instruction-solution pairs for scheduling problems.
	ECLIPSE [85]	NAACL, 2024	Prompt-based	Apply iterative optimization to the design of jailbreak attacks.
	LLOME [93]	arXiv, 2024	Learning-based	Use preference learning to satisfy complex biophysical constraints.
	LLMDSM [82]	arXiv, 2024	Prompt-based	Apply iterative optimization to design-structure-matrix sequencing.
	LMCO [83]	WCL, 2024	Prompt-based	Apply iterative optimization to wireless-network design.
Low-level LLMs for Optimization Algorithms	MGSCO [90]	arXiv, 2025	Prompt-based	Use multimodal LLMs to interpret visual representations of abstract graphs.
	ORFS [84]	arXiv, 2025	Prompt-based	Apply iterative optimization to automated parameter tuning in chip design.
	LMX [103]	TELO, 2023	Operators	Use LLMs as operators for crossover and recombination of textual genomes.
	GPT-NAS [96]	arXiv, 2023	Initialization	Use prior knowledge from LLMs to initialize neural architecture search.
	LMEA [104]	CEC, 2023	Operators	Use LLMs as crossover, mutation, and selection operators in EAs.
	LLM-PP [97]	arXiv, 2023	Initialization	Use LLM-based performance prediction to support initialization.
	LMOEA [108]	EMO, 2023	Operators	Use zero-shot LLM prompting as a search operator within MOEA/D.
	OPLLM [99]	AI, 2023	Initialization	Assist non-expert users in initializing financial plans.
	LEO [107]	Neucom, 2024	Operators	Guide candidates from separate exploration and exploitation pools with LLMs.
	LLM-GA [98]	BiB, 2024	Initialization	Use LLMs to initialize high-quality mutant pools for enzyme-design GAs.
	GE [189]	GECCO, 2024	Operators	Use role-based prompts to increase creativity and diversity in LLM-assisted NAS.
	LLMAES [110]	ICIC, 2024	Operators	Generate selected population candidates with LLMs to reduce interaction costs.
	LLMTES [117]	arXiv, 2024	Configuration	Tune evolution strategies sequentially through LLM-based feedback.
	LAEA [122]	SWEVO, 2024	Evaluation	Reformulate model-assisted selection as classification and regression.
	LICO [123]	arXiv, 2024	Evaluation	Use LLMs as surrogate models for molecular-science applications.
	LLMSM [124]	CAI, 2024	Evaluation	Coordinate model selection and training for engineering optimization with multiple LLMs.
	LLMCES [119]	GECCO, 2024	Configuration	Control evolution-strategy step sizes with an OPRO-like mechanism.
	AKFLM [102]	arXiv, 2024	Operators	Support mutation and other generative stages in evolutionary multitask optimization with LLMs.
	LLMAMO [111]	arXiv, 2024	Operators	Invoke LLMs to generate elite solutions when population improvement is insufficient.
	LTC [118]	ESANN, 2024	Configuration	Control CMA-ES dynamically through sequential LLM feedback.
	LLMEVO [100]	arXiv, 2025	Operators	Evaluate LLMs in selection, crossover, and mutation, while identifying limitations in initialization.
	LLM-GE [187]	GECCO, 2025	Operators	Use LLMs as crossover and mutation operators for YOLO architecture optimization.
	LAOS [120]	GECCO, 2025	Configuration	Replace optimization trajectories with state features for LLM-based operator selection.
	PAIR [105]	arXiv, 2025	Operators	Use LLMs primarily as selection operators to extend LMEA.
	LLMMS [126]	arXiv, 2025	Evaluation	Use LLM meta-surrogates and token-sequence representations for cross-task knowledge transfer.
	LMPSO [106]	arXiv, 2025	Operators	Simulate PSO dynamics with LLMs and adapt LMEA to specific algorithms.
	LLM-SAEA [125]	arXiv, 2025	Evaluation	Select surrogate models and infill criteria dynamically with LLMs.
High-level LLMs for Optimization Algorithms	LLMGM [134]	GECCO, 2023	Single-step	Decompose and recombine six metaheuristics to generate hybrid algorithms in a single step.
	FunSearch [136]	Nature, 2023	Iterative	Generate code fragments with LLMs and use EAs to search the resulting function space.
	AEL [137]	arXiv, 2023	Iterative	Extend FunSearch with heuristic principles and improve performance on TSP instances.
	AS-LLM [131]	IJCAI, 2023	Selection	Extract high-dimensional algorithm features from code and text for algorithm selection.
	EoH [29]	ICML, 2024	Iterative	Co-evolve heuristic descriptions and code through predefined crossover and mutation prompts.
	ReEvo [159]	NeurIPS, 2024	Iterative	Guide hyper-heuristic search through reflective evolution with LLMs.
	AutoSAT [153]	arXiv, 2024	Iterative	Combine multiple heuristic strategies to guide LLM-based algorithm generation.
	LLaMEA [121]	TEVC, 2024	Iterative	Use LLMs within evolutionary mutation and selection to generate heuristics.
	L-AutoDA [145]	GECCO, 2024	Iterative	Apply AEL to generate adversarial-attack algorithms for cybersecurity.
	MOELLM [154]	TEVC, 2024	Iterative	Combine robust testing with dynamic selection to generate multi-objective optimization algorithms.
	LLMEPS [155]	PPSN, 2024	Iterative	Establish a baseline for automated algorithm design based on EoH and ReEvo.
	EvoCAF [144]	PPSN, 2024	Iterative	Apply EoH to evolve cost-aware acquisition functions for Bayesian optimization.
	TS-EoH [143]	arXiv, 2024	Iterative	Apply EoH to algorithm generation for edge-server task scheduling.
	MEoH [141]	AAAI, 2024	Iterative	Extend EoH to multi-objective search with additional algorithm-quality criteria.
	LLaMEA-HPO [150]	TELO, 2024	Iterative	Integrate hyperparameter optimization into the LLaMEA search cycle.
	CMLLM [156]	EvoApps, 2024	Iterative	Control LLM mutation through dynamic prompts to improve LLaMEA.
	HSEvo [161]	AAAI, 2024	Iterative	Combine harmony search with GAs and optimize portfolio quality and diversity.
	LLM4AD [138]	arXiv, 2024	Iterative	Provide an EoH-based platform for heuristic algorithm design.
	MCTS-AHD [160]	arXiv, 2025	Iterative	Organize generated heuristics in a tree and explore them with MCTS.
	CEG [147]	GECCO, 2025	Iterative	Analyze LLM-generated code and its evolutionary dynamics during search.
	OPSLAD [152]	GECCO, 2025	Iterative	Apply LLaMEA to industrial photonic-structure optimization.
	BLADE [149]	GECCO, 2025	Iterative	Establish a standardized and reproducible evaluation framework for LLM-driven algorithm generation (LLaMEA specifically).
	AutoHD [158]	arXiv, 2025	Iterative	Discover heuristics for complex planning tasks under LLM guidance.
	LAMA [157]	TEVC, 2025	Iterative	Construct an automated memetic algorithm with LLM-designed heuristics.
	LAS [139]	arXiv, 2025	Iterative	Analyze the fitness landscapes of LLM-assisted algorithm search.
	LLaMEA-BO [151]	arXiv, 2025	Iterative	Apply LLaMEA to automated Bayesian-optimization algorithm design.
	InstSpecHH [132]	arXiv, 2025	Selection	Use LLM semantic reasoning for context-aware algorithm selection.
	BSALMD [148]	arXiv, 2025	Iterative	Analyze behavioral spaces to characterize algorithm-evolution trajectories.
	FLAAD [140]	arXiv, 2025	Iterative	Fine-tune LLMs for algorithm generation with diversity-aware ranked sampling.
	EoH-S [142]	arXiv, 2025	Iterative	Extend EoH to evolve complementary portfolios of algorithms.

C REPRESENTATIVE BENCHMARKS

As discussed in the main text, an effective benchmark serves two purposes. It provides a diverse problem set that exposes the strengths and limitations of competing methods, and it supports centralized evaluation under comparable conditions. Table IV summarizes representative benchmarks for optimization modeling and solving. For each benchmark, we report the venue, scale, problem scope, and construction strategy. The modeling benchmarks progress from competition-derived datasets and manual curation to scalable synthetic generation. The solving benchmarks range from single-turn graph and path-planning tasks to agentic suites for iterative algorithm design. Together, they reflect the field’s transition from direct problem solving to the generation and evaluation of optimization algorithms.

TABLE IV
REPRESENTATIVE BENCHMARKS FOR LLM-DRIVEN OPTIMIZATION MODELING AND SOLVING.

	Benchmark	Venue	Scale	Technical Summary
Modeling	NL4Opt [40]	EMNLP, 2022	245	Competition-derived benchmark for translating natural language into LP models.
	NLP4LP [52]	ICML, 2024	67	Natural-language descriptions of LP and MILP models.
	NL2OPT [54]	INFOR, 2024	70	Handcrafted LP, MILP, and QP instances with structured specifications and Zimpl ground truth.
	ComplexOR [50]	ICLR, 2024	37	Complex MILP problems that require multistep formulation.
	MAMO [62]	NAACL, 2025	1,209	Two difficulty levels spanning LP, MILP, and ordinary differential equations.
	IndustryOR [27]	OR, 2025	100	Synthetically generated industrial problems covering LP, IP, MILP, and NLP.
	OptiBench [72]	ICLR, 2025	605	Synthetic LP, IP, MILP, and NLP instances with controllable complexity and verified equivalence.
	DP-Bench [74]	arXiv, 2025	132	Textbook-derived deterministic and stochastic dynamic-programming problems with finite or infinite horizons.
	DCP-Bench [162]	ECAI, 2025	164	Natural-language CSP and COP instances with constraint-programming ground truth.
	OptMath [73]	ICML, 2025	165	Scalable bidirectional synthesis across LP, IP, MILP, NLP, and SOCP formulations.
Solving	NLGraph [166]	NeurIPS, 2023	29,370	Eight polynomial-time graph-reasoning tasks for systematic LLM evaluation.
	PPNL [167]	ICLR, 2024	160,000+	Controllable grid-world path-planning tasks for spatial and temporal reasoning.
	GraphArena [168]	ICLR, 2025	10,000	Real-world graph benchmark covering both P-class and NP-complete tasks.
	ALE-Bench [173]	NeurIPS, 2025	–	Long-horizon, objective-driven algorithm engineering with Elo-style scoring.
	OPT-BENCH [169]	arXiv, 2025	30	Large-search-space benchmark with an OPT-Agent for Kaggle ML tasks and NP problems.
	CO-Bench [170]	AAAI, 2026	6,482	Benchmark for LLM-based algorithm search across 36 real-world CO problems in eight categories.
	FrontierCO [171]	ICLR, 2026	258+	Competition-scale TSP, CVRP, and MIS instances with easy and hard splits.
	HeuriGym [172]	ICLR, 2026	9	Low-exposure CO problems for evaluating LLM-generated heuristics with the Quality-Yield Index.
	LLM4AD [138]	arXiv, 2026	160+	Unified platform for LLM-based algorithm design with modular search, sandboxing, and a GUI.

D DETAILED EVALUATION OF OPTIMIZATION MODELING METHODS

To support the empirical synthesis in the main text, we evaluate a broad set of baselines for optimization modeling. Table V reports the complete performance matrix. The baseline category includes the general-purpose models GPT-3.5-Turbo, GPT-4, GPT-4o, and DeepSeek-V3, together with the reasoning models DeepSeek-R1, OpenAI-o1, and OpenAI-o3. The prompt-based category includes CoE [50], CoT [174], CAFA [59], OptiMUS [51], LLM-MCTS [60], StepORLM+GenPRM [80], and OptiTree [163]. The learning-based category includes ORLM [27], OptMATH [73], LLMOPT [77], StepORLM [80], OptiTrust [71], Step-Opt [76], SIRL [78], DeepOR [164], and OR-R1 [253]. The main text aggregates 11 representative methods for the average-accuracy analysis, whereas this supplementary document reports all available results at the dataset level.

We collect the results through a three-stage protocol. First, we extract metrics reported in the original publications. Second, we incorporate values from official benchmark reports after cross-checking the method and metric definitions. Third, we fill remaining gaps with results from credible reproduction studies, including the reproductions reported by OptiTree. The evaluation environments are not fully homogeneous. They differ in solver configurations, few-shot examples, decoding temperatures, and the number of trials used to estimate pass@1. Consequently, cross-paper comparisons require caution. We reduce this risk by prioritizing original and benchmark-reported values and by aligning metric definitions whenever possible.

Table V reports results for all 23 methods, while Fig. 13(a) and (b) visualize the same data as a performance heatmap and category-level trajectories. The overall ordering is consistent with the main-text analysis. Learning-based methods perform best on average, frontier reasoning models follow closely, and prompt-based methods show the greatest internal variation. On easier datasets, such as NL4Opt and MAMO-E, all three categories approach saturation and their performance differences remain small. The categories separate on moderately difficult datasets, including NLP4LP and OptiBench, where stronger prompt-based and learning-based methods outperform weaker baselines. The largest gaps appear on MAMO-C, ComplexOR, IndustryOR, and OptMath. On these datasets, weaker baselines deteriorate sharply, whereas learning-based and search-augmented prompt-based methods retain substantially higher accuracy. Strong reasoning models, particularly DeepSeek-R1, OpenAI-o1, and OpenAI-o3, remain competitive on the difficult datasets and narrow the advantage of specialized learning-based methods. This trend also challenges conventional prompt orchestration. As the reasoning capabilities of base models improve, future gains are more likely to come from structured multi-agent interaction and semantic-space search than from standard prompting alone.

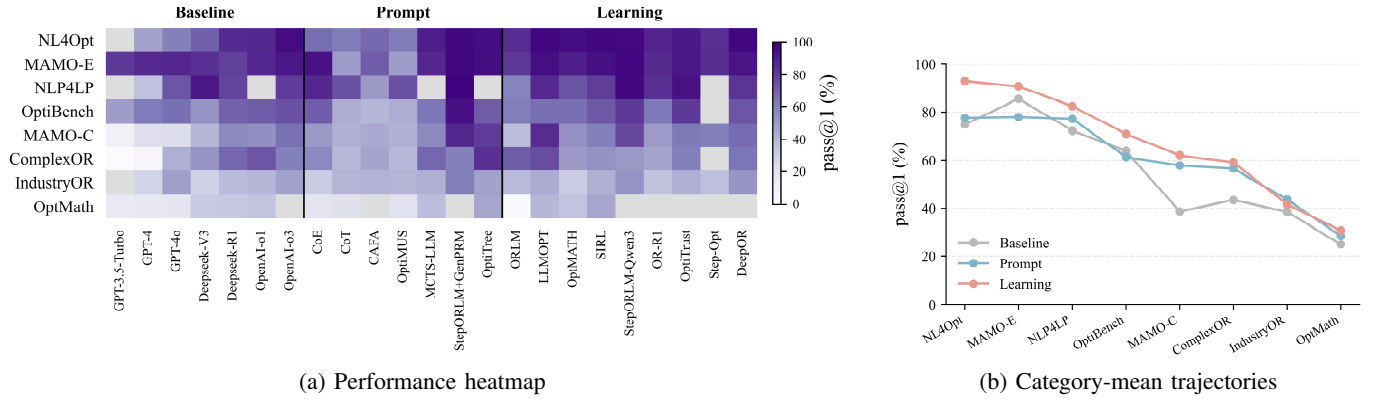


Fig. 13. Detailed performance on eight optimization-modeling benchmarks. (a) Per-dataset pass@1 for all 23 methods, with datasets ordered by increasing difficulty; gray cells indicate unreported results. (b) Category-mean pass@1 over the same dataset order; the curves converge on easier datasets and separate as difficulty increases.

TABLE V

PERFORMANCE ON OPTIMIZATION-MODELING BENCHMARKS. ALL VALUES ARE ACCURACY (%). * DENOTES VALUES REPORTED IN THE ORIGINAL PAPERS, † DENOTES VALUES REPORTED IN BENCHMARK STUDIES, AND UNMARKED VALUES COME FROM CREDIBLE REPRODUCTION STUDIES. THE BEST RESULT WITHIN EACH METHOD CATEGORY IS BOLD FACED.

	Method	NL4Opt	MAMO-E	MAMO-C	NLP4LP	ComplexOR	IndustryOR	OptiBench	OptMath	Avg.
Baseline	GPT-3.5-Turbo	–	81.3 [†]	9.5 [†]	–	0.5 [†]	–	49.1 [†]	15.0 [†]	–
	GPT-4	47.3	86.5 [†]	21.1 [†]	35.8 [†]	4.9 [†]	28.0 [†]	62.8 [†]	16.6 [†]	37.9
	GPT-4o	61.2	87.3 [†]	22.8 [†]	73.6	42.9	48.4	66.1 [†]	17.5	52.5
	DeepSeek-V3	70.5	84.3	39.8	92.1	52.6	29.0	52.4	32.6 [†]	56.7
	DeepSeek-R1	86.1	79.5	57.3	78.6	68.4	38.0	70.2	33.1	63.9
	OpenAI-o1	87.1	87.6	54.5	–	73.6	40.0	71.5	34.9	–
	OpenAI-o3	96.2	92.4	65.8	81.0	60.7	47.8	74.8	–	–
Prompt-based	Chain-of-Experts	66.7	94.4	50.6	87.4	57.1	31.2	71.2	18.6	59.7
	Chain-of-Thought	62.2	49.5	42.3	74.7	39.2	40.5	43.6	20.5	46.6
	CAFA	68.1	71.2	44.5	50.0	46.4	41.1	40.1	–	–
	OptiMUS	62.2	49.5	42.3	74.7	39.2	40.5	43.6	20.2 [†]	46.5
	MCTS-LLM	90.3	87.4	56.8	–	68.4	42.0	64.0	37.3	–
	StepORLM+GenPRM	97.2*	97.8*	87.4*	98.9*	61.1*	61.9*	94.6*	–	–
	OptiTree	96.2*	95.6*	81.0*	–	84.2*	48.0*	71.9*	45.8*	–
Learning-based	ORLM-LLaMA-3-8B	85.7*	82.3*	37.4*	59.5	71.4	38.0*	61.8	2.6 [†]	54.8
	LLMOPT-Qwen2.5-14B	97.3*	95.3*	85.8*	86.5*	76.5*	44.0*	66.4*	40.0*	74.0
	OptMATH-Qwen2.5-32B	95.9*	89.9*	54.1*	73.9	50.0	31.0*	66.1*	34.7*	62.0
	SIRL-Qwen2.5-32B	98.0*	94.6*	61.1*	80.6	53.6	42.0*	72.6	45.8*	68.5
	StepORLM-Qwen3-8B	96.7*	97.6*	77.5*	97.2*	50.0*	52.4*	81.9*	–	–
	OR-R1-Qwen3-8B	88.3*	86.1*	49.9*	84.6*	46.3*	35.3*	62.9*	–	–
	OptiTrust-Granite3.2-8B	91.6*	92.3*	63.1*	94.4*	61.1*	42.9*	81.4*	–	–
	Step-Opt-LLaMA-3-8B	84.5*	85.3*	61.6*	–	–	36.4*	–	–	–
	DeepOR-Qwen3-8B	97.7*	93.2*	67.1*	82.9*	64.3*	52.2*	73.8*	–	–

A second finding concerns evaluation coverage. Many entries in Table V are unreported, and the missing values are concentrated on the most difficult datasets and among the strongest methods. These methods are often evaluated only on training-aligned or author-selected subsets. Consequently, headline scores are not directly comparable, and strong results on partial subsets may not generalize. Future benchmark research should address this limitation in two ways. First, the community needs an agentic optimization-modeling platform that evaluates common prompt-based and learning-based methods under a shared protocol. Such a platform would enable direct comparison and independent validation of partial-coverage claims. Second, prompt-based methods should be re-evaluated with newer and stronger base models. Most existing studies use GPT-4o or earlier models, so evaluations on frontier backbones are necessary to determine the remaining contribution of prompt-level strategies.

E

DETAILED EVALUATION OF LLMs AS OPTIMIZERS

To support the empirical claims in the main text, we report complete per-function results for the LLM-as-optimizer study in Table VI. The classical group contains four established black-box optimizers: DE, PSO, CMA-ES, and random search.

TABLE VI

PER-FUNCTION RELATIVE IMPROVEMENT r (MEAN $\pm$ STANDARD DEVIATION OVER FIVE SEEDS) ON FOUR BBOB FUNCTIONS AT DIMENSIONS 2, 5, AND 10. CLASSICAL EAS AND LLM OPTIMIZERS USE THE SAME BUDGET OF 1608 FUNCTION EVALUATIONS. THE BEST VALUE IN EACH METHOD CATEGORY, CLASSICAL OR LLM-BASED, IS BOLDFACED IN EACH COLUMN.

Dim	Method	F1 (Sphere)	F8 (Rosenbrock)	F11 (Discus)	F20 (Schwefel)	Avg.
2	CMA-ES	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.945 $\pm$ 0.075	0.986
	PSO	1.000 $\pm$ 0.000	0.999 $\pm$ 0.002	1.000 $\pm$ 0.000	0.871 $\pm$ 0.127	0.968
	DE	0.999 $\pm$ 0.003	0.963 $\pm$ 0.068	1.000 $\pm$ 0.000	0.895 $\pm$ 0.151	0.964
	Random	0.859 $\pm$ 0.282	0.970 $\pm$ 0.039	0.999 $\pm$ 0.001	0.782 $\pm$ 0.387	0.902
	GPT-5-mini	0.999 $\pm$ 0.001	0.932 $\pm$ 0.041	0.917 $\pm$ 0.186	0.736 $\pm$ 0.251	0.896
	DeepSeek-V4	0.916 $\pm$ 0.140	0.794 $\pm$ 0.396	1.000 $\pm$ 0.000	0.761 $\pm$ 0.201	0.868
	Qwen3.5	0.265 $\pm$ 0.411	0.680 $\pm$ 0.400	0.913 $\pm$ 0.193	0.616 $\pm$ 0.382	0.619
	CMA-ES	1.000 $\pm$ 0.000	0.998 $\pm$ 0.004	0.994 $\pm$ 0.012	1.000 $\pm$ 0.001	0.998
	PSO	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.985 $\pm$ 0.033	0.996
	DE	0.887 $\pm$ 0.071	0.974 $\pm$ 0.036	1.000 $\pm$ 0.000	0.994 $\pm$ 0.012	0.964
5	Random	0.775 $\pm$ 0.141	0.870 $\pm$ 0.245	0.996 $\pm$ 0.007	0.998 $\pm$ 0.002	0.910
	GPT-5-mini	0.361 $\pm$ 0.156	0.710 $\pm$ 0.224	0.786 $\pm$ 0.299	0.866 $\pm$ 0.168	0.681
	DeepSeek-V4	0.795 $\pm$ 0.301	0.897 $\pm$ 0.130	0.851 $\pm$ 0.313	0.993 $\pm$ 0.011	0.884
	Qwen3.5	0.258 $\pm$ 0.244	0.600 $\pm$ 0.413	0.803 $\pm$ 0.440	0.673 $\pm$ 0.311	0.583
	CMA-ES	1.000 $\pm$ 0.000	0.999 $\pm$ 0.001	0.991 $\pm$ 0.018	1.000 $\pm$ 0.000	0.997
	PSO	0.972 $\pm$ 0.037	0.999 $\pm$ 0.001	1.000 $\pm$ 0.001	0.979 $\pm$ 0.046	0.987
	DE	0.663 $\pm$ 0.129	0.876 $\pm$ 0.149	1.000 $\pm$ 0.000	0.991 $\pm$ 0.007	0.883
	Random	0.648 $\pm$ 0.106	0.914 $\pm$ 0.087	0.992 $\pm$ 0.012	0.903 $\pm$ 0.144	0.864
	GPT-5-mini	0.200 $\pm$ 0.159	0.444 $\pm$ 0.313	0.910 $\pm$ 0.169	0.509 $\pm$ 0.070	0.516
	DeepSeek-V4	0.630 $\pm$ 0.164	0.865 $\pm$ 0.221	0.988 $\pm$ 0.026	0.837 $\pm$ 0.177	0.830
10	Qwen3.5	0.146 $\pm$ 0.200	0.587 $\pm$ 0.413	0.802 $\pm$ 0.413	0.580 $\pm$ 0.391	0.529

All four are tensorized in the EvoX framework [254]. The LLM group follows the OPRO paradigm and iteratively generates raw floating-point candidate vectors at a sampling temperature of 1.0. We evaluate GPT-5-mini, DeepSeek-V4, and Qwen3.5. Chain-of-thought reasoning is disabled because the task requires direct numerical candidate generation rather than extended verbal reasoning. GPT-5-mini uses its minimum reasoning-effort setting, while the explicit thinking modes of DeepSeek-V4 and Qwen3.5 are disabled.

Both groups receive the same evaluation budget. Each method evaluates eight candidates per iteration for 200 iterations, in addition to the eight initial candidates, for a total of 1608 function evaluations per run. The test suite contains four BBOB functions [175] with distinct landscape properties: Sphere, Rosenbrock, Discus, and Schwefel. We evaluate dimensions 2, 5, and 10 with five independent random seeds for each configuration. Performance is measured by the relative improvement r , defined as

$$r = \text{clip}_{[0,1]} \left(\frac{f_{\text{init}} - f_{\text{best}}}{f_{\text{init}} - f_{\text{opt}}} \right), \quad (1)$$

where f_{init} is the best fitness among the initial random samples, f_{best} is the best fitness obtained by the end of the run, and f_{opt} is the known optimum of the corresponding BBOB instance. We use this scaled metric because the benchmark functions operate on markedly different numerical scales. Typical values are on the order of 10^3 for Sphere and 10^5 for Discus, and the instance optima also vary across dimensions. Raw fitness values are therefore not comparable across functions or dimensions. In contrast, r measures the fraction of the initial-to-optimal gap closed by an optimizer. The normalization removes both function-specific scale and dimension-specific optimum effects, which permits the cross-function and cross-dimension averages reported in the main text. Clipping the metric to $[0, 1]$ assigns zero improvement to runs that do not improve on the initialization and prevents minor numerical overshoots beyond the known optimum from distorting the averages.

Table VI reports the relative improvement of all seven methods across the three dimensions. The classical optimizers remain consistently strong. CMA-ES achieves average improvements of 0.986, 0.998, and 0.997 at dimensions 2, 5, and 10, respectively, while every classical method remains above 0.86 on average at each dimension. The LLM optimizers perform substantially worse overall, and their performance generally declines as the dimension increases. This weakness is visible even on Sphere, which the classical methods solve reliably: at dimension 10, GPT-5-mini and Qwen3.5 attain only 0.200 and 0.146, respectively. Their stronger results on Discus at the same dimension show that landscape structure also affects performance, but increasing dimension remains a major source of degradation. No LLM dominates across all settings. GPT-5-mini achieves the highest LLM average at dimension 2, whereas DeepSeek-V4 leads at dimensions 5 and 10. DeepSeek-V4 also shows a nonmonotonic

TABLE VII
COMPUTATIONAL COST AND FAILURE STATISTICS FOR EACH LLM IN THE OPTIMIZER STUDY, AGGREGATED OVER 60 RUNS PER MODEL.

Method	API calls	Input (M)	Output (M)	Wall-clock/run (s)	Fallback events	Stagnant runs (/60)
Classical EAs	—	—	—	~ seconds	0	0
GPT-5-mini	12000	14.10	2.91	1785	11	5
DeepSeek-V4	12000	13.65	2.83	694	56	1
Qwen3.5	12000	18.15	3.74	882	690	16
LLM total	36000	45.90	9.48	—	757	22

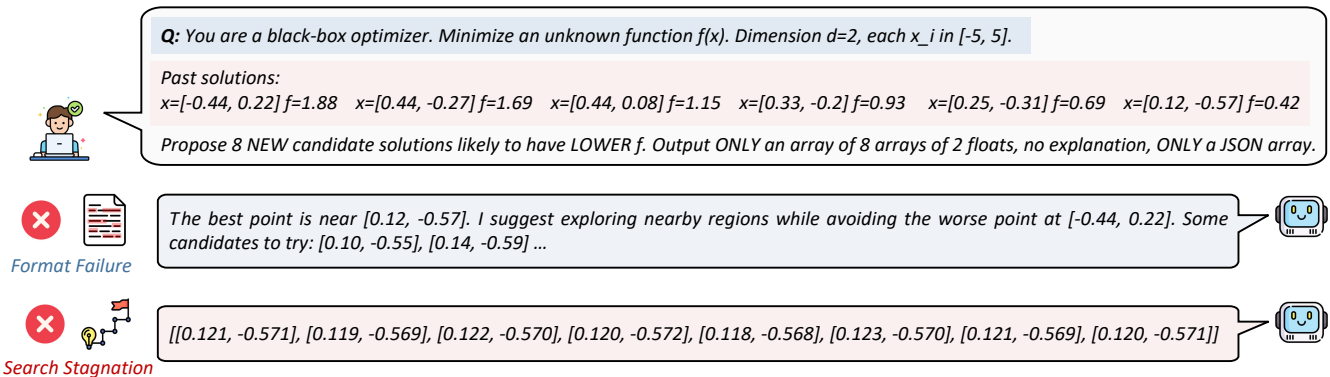


Fig. 14. Representative examples of the two main LLM-as-optimizer failure modes: format failure and search stagnation.

trend, with averages of 0.868, 0.884, and 0.830 across the three dimensions. These results support the main-text conclusion that current LLM-driven iterative optimization does not match mature evolutionary algorithms on continuous problems and scales poorly with dimension. The pattern suggests that limited numerical-search capability, rather than any single landscape property, is the principal bottleneck.

Table VII summarizes computational overhead and model failures. Each LLM run requires 200 sequential API calls. The mean wall-clock time per run ranges from 694 seconds for DeepSeek-V4 to 1785 seconds for GPT-5-mini, whereas a classical optimizer completes a run within a few seconds on a standard local device. Token consumption is also substantial. Across 60 runs per model, the three LLMs consume 45.90 million input tokens and 9.48 million output tokens. Input tokens dominate because OPRO appends the cumulative optimization trajectory to the prompt at every iteration.

Two recurrent failure modes appear during search, and Fig. 14 illustrates representative examples. A format failure occurs when the model output cannot be parsed as the required candidate array. The system then invokes a perturbation-based fallback. Table VII records 11 such events for GPT-5-mini, 56 for DeepSeek-V4, and 690 for Qwen3.5. Search stagnation is the second failure mode. It occurs when the model returns valid candidates but fails to improve the objective. We classify a run as stagnant when its final relative improvement satisfies $r < 0.2$. Under this criterion, 16 of the 60 Qwen3.5 runs are stagnant, compared with five GPT-5-mini runs and one DeepSeek-V4 run. The clearest example occurs on two-dimensional Sphere, where four of the five Qwen3.5 runs plateau near their initial fitness without triggering a fallback. This behavior is not unique to the OPRO implementation. It indicates a broader tendency of current foundation models to favor conservative local exploitation over effective exploration in continuous floating-point spaces. Section F examines the same limitation in the context of dynamic parameter control.

F

DETAILED EVALUATION OF LOW-LEVEL ASSISTANCE

To support the empirical claims in the main text, we examine dynamic control of the step size in a (1+1)-ES. We compare LLM-based controllers with representative methods from the MetaBBO family [12], which is the most extensively studied family of learned optimizers in this setting. The trained controllers include reinforcement-learning methods [16] and an evolution-strategy-based method [180]. DDQN [179] and PPO [12] represent the value-based and policy-based branches of MetaBBO-RL, respectively.

All trained and LLM-based controllers receive the same state representation and reward signal. The state contains the remaining evaluation budget, the number of generations since the last improvement, the relative fitness improvement achieved so far, the recent offspring-acceptance rate, the current step size, the best solution, and its fitness. These descriptors are widely used in dynamic algorithm configuration [255] and capture information from the optimization process, solution space, and

TABLE VIII
PER-FUNCTION RELATIVE IMPROVEMENT r (MEAN OVER FIVE SEEDS) FOR STEP-SIZE CONTROL IN A (1+1)-ES AT DIMENSION 20 WITH A BUDGET OF 1000 FUNCTION EVALUATIONS. THE CONTROLLER UPDATES THE STEP SIZE EVERY 20 GENERATIONS. THE BEST RESULT WITHIN EACH METHOD CATEGORY IS BOLDFACED.

Method	Rastrigin	Ackley	Griewank	Schwefel	Avg.
fixed $\sigma=0.8$	0.605	0.031	0.269	0.762	0.417
DDQN	0.626	0.080	0.404	0.658	0.442
PPO	0.467	0.021	0.312	0.572	0.343
OpenAI-ES	0.225	0.039	0.144	0.437	0.211
GPT-5-mini	0.607	0.039	0.560	0.774	0.495
DeepSeek-V4	0.640	0.043	0.596	0.731	0.503
Qwen3.5	0.658	0.041	0.574	0.771	0.511

fitness landscape. All controllers choose from the same discrete action space. We conduct the experiments at dimension 20 with a budget of 1000 function evaluations and update the step size every 20 generations.

Table VIII reports the relative improvement of the seven controllers. All three LLM controllers outperform the fixed- σ baseline and the trained MetaBBO controllers on average. Qwen3.5 achieves the highest mean score of 0.511, followed by DeepSeek-V4 at 0.503 and GPT-5-mini at 0.495; each exceeds the fixed baseline of 0.417. DDQN is the strongest MetaBBO controller with an average of 0.442, but it remains below every LLM controller. PPO achieves 0.343, while OpenAI-ES ranks last at 0.211 and does not transfer effectively across the test functions. The LLM advantage is not confined to one landscape. On Rastrigin, Griewank, and Schwefel, the three LLMs achieve ranges of 0.61–0.66, 0.56–0.60, and 0.73–0.77, respectively, and outperform the strongest MetaBBO controller on each function. Ackley is the only exception. Every controller remains below 0.08 because a (1+1)-ES with one offspring per iteration cannot reliably escape its many local optima within the 1000-evaluation budget. No LLM is uniformly best. Qwen3.5 leads on Rastrigin, Schwefel, and the overall average, whereas DeepSeek-V4 leads on Ackley and Griewank. These results support the main-text finding that general LLM reasoning can match or exceed trained MetaBBO controllers on this single-parameter control task, particularly when the trained policies do not transfer across landscapes.

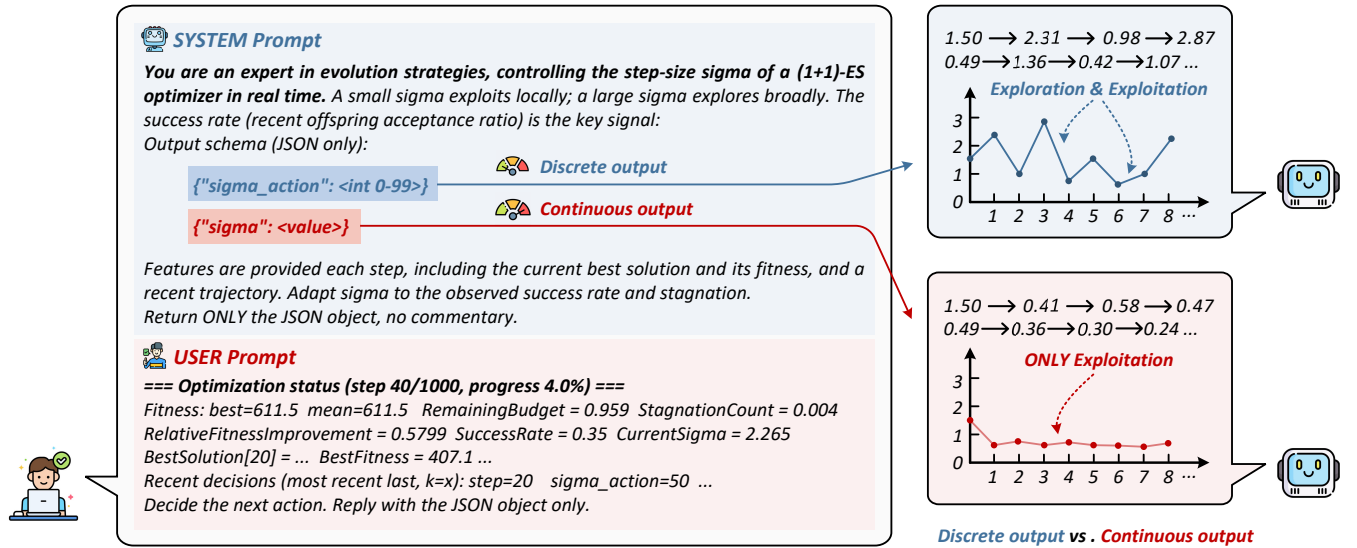


Fig. 15. Representative step-size trajectories under the two output paradigms for low-level LLM assistance: direct continuous output and structured discrete output.

Beyond controller performance, we examine how output representation affects LLM-based parameter control. Figure 15 compares two paradigms derived from prior work [119], [120]. The first follows the OPRO format: it provides the full optimization trajectory and asks the LLM to output a continuous step size. The second provides a structured MetaBBO-style state and asks the LLM to select a discrete action. On Rastrigin, direct continuous output confines the step size to a narrow range and produces predominantly exploitative behavior. Structured discrete output supports more flexible transitions between exploration and exploitation. The output representation is a primary source of this difference. Continuous generation requires the model to reason directly over floating-point values, which reproduces the limitation observed in the LLM-as-

optimizer experiment. Discrete actions reduce the decision space and therefore simplify control. The structured state also avoids the attention dispersion and context growth caused by an ever-expanding optimization trajectory. Recent work [256] further suggests that effective parameter control should move from population-level to individual-level decisions. For LLM controllers, a promising implementation is to assign parameters to buckets of batched individuals.

G

DETAILED EVALUATION OF HIGH-LEVEL ASSISTANCE

To evaluate high-level algorithm generation in greater detail, we compare results reported by three recent comprehensive benchmarks: CO-Bench [170], HeuriGym [172], and FrontierCO [171]. Each benchmark uses its own standardized protocol, so the comparison is intended to reveal broad regime-level trends rather than establish a single unified ranking. The benchmarks are complementary. CO-Bench provides broad coverage of classical combinatorial-optimization problems and LLM-based methods. HeuriGym focuses on low-exposure scientific and engineering problems and uses the Quality-Yield Index to distinguish solution validity from quality. FrontierCO evaluates competition-scale real-world instances that are orders of magnitude larger. Comparing the reported results across these benchmarks shows how the apparent advantage of LLM-based methods changes as problem realism and evaluation rigor increase.

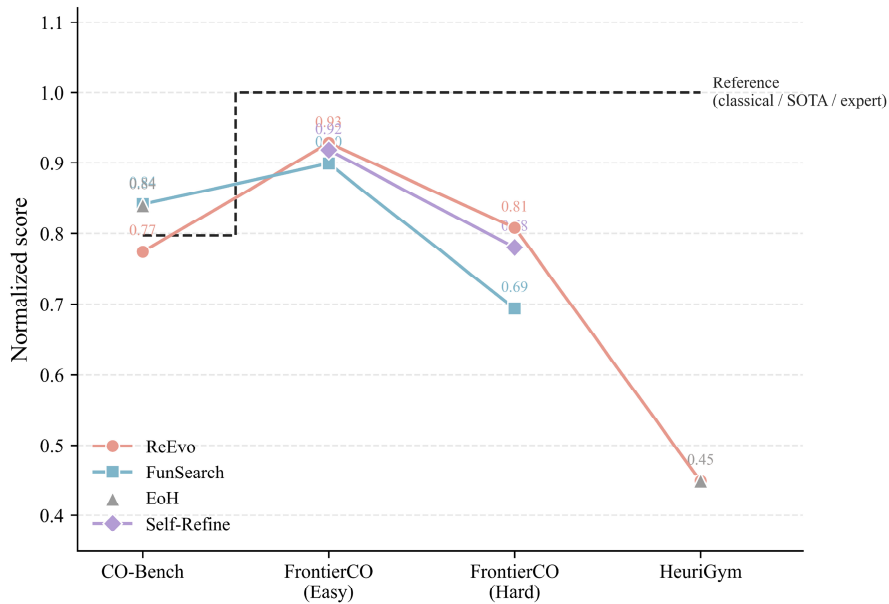


Fig. 16. Performance of representative LLM-based methods across problem regimes of increasing realism. The dashed line denotes the reference baseline: a classical solver on CO-Bench and the state-of-the-art or expert baseline on the other benchmarks. Values above the line exceed the corresponding reference, while values below it do not. Scores are normalized so that 1.0 matches the best-known, state-of-the-art, or expert solution. Missing markers indicate that a method was not evaluated in that regime.

Figure 16 shows a consistent decline as the evaluation regime becomes more realistic. On CO-Bench, FunSearch and EoH both achieve normalized scores of 0.84 and slightly exceed the classical-solver reference of 0.797, while ReEvo scores 0.77. Thus, high-level generation frameworks can rival the classical baseline on standard benchmark problems. This advantage disappears on more demanding regimes. On the FrontierCO easy set, the evaluated methods already fall below the state-of-the-art reference; FunSearch scores 0.90. On the hard set, its score falls further to 0.69. HeuriGym reveals the largest deficit: EoH and ReEvo each score 0.45, and even Gemini-2.5-Pro reaches only 0.62 against the expert reference. ReEvo is the only method evaluated in all four regimes, and its score declines from 0.77 to 0.45. Differences in reference strength explain part of this reversal. CO-Bench uses a time-limited classical solver, whereas FrontierCO and HeuriGym compare against state-of-the-art or expert performance. Exceeding the CO-Bench baseline is therefore a less demanding criterion. However, the reference alone does not explain the full decline. The methods also lose feasibility, scalability, and novelty as task realism increases.

Three recurring failure modes help explain this decline, and Fig. 17 illustrates them through representative agent dialogues. First, iterative frameworks often modify the same flawed program without correcting its central defect. HeuriGym reports that broken context across iterations and poorly integrated execution feedback lead agents to rename variables or reorder unrelated loops, and agents may even return to previously failed versions. Second, agents cannot reliably assess the quality of the algorithms they generate. FrontierCO identifies this weakness as a major source of variance: an agent may describe an algorithm as near-optimal and efficient even when it times out on one routing instance, yet the same procedure may outperform the state of the art on another. The agent cannot distinguish these outcomes in advance. Third, feasibility and efficiency are

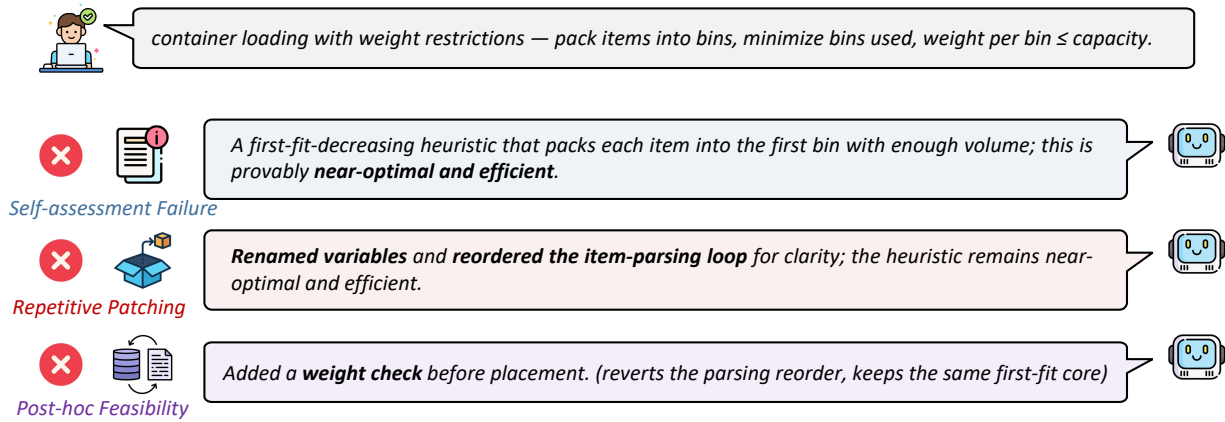


Fig. 17. Representative high-level algorithm-generation failure on a container-loading task with weight constraints.

often checked only after generation. Some heuristics violate constraints during construction, while others use local-search depths with exponential cost and therefore time out on large instances. As a result, even the strongest agents produce valid solutions less consistently than the classical solver. These failures identify the central research frontier: reliable self-assessment, cumulative use of feedback, and constraint-aware reasoning.